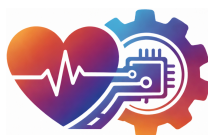




**UNIVERSIDAD
DE BURGOS**

Escuela Politécnica Superior
Grado en Ingeniería de la Salud



**TFG del Grado en Ingeniería de la
Salud**

**Aplicación web de cálculo de
indicadores en la URCCPQ**

Presentado por Nicolás José García Gómez
en Universidad de Burgos

8 de junio de 2026

Tutores: David García García – Cristina Arlanzón Pérez



**UNIVERSIDAD
DE BURGOS**

Escuela Politécnica Superior

Grado en Ingeniería de la Salud

D. David García García, profesor del departamento de Ingeniería Informática del área de Lenguajes y Sistemas Informáticos. D. Cristina Arlanzón Pérez, Jefa de la Unidad de Reanimación y Cuidados Críticos postquirúrgicos del Servicio de Anestesiología y Reanimación del Hospital Universitario de Burgos.

Expone/n:

Que el/la alumno D. Nicolás José García Gómez, con DNI 71707732B, ha realizado el Trabajo final de Grado en Ingeniería de la Salud titulado Aplicación web de cálculo de indicadores en la URCCPQ.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección los que suscriben, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, a 8 de junio de 2026

Vº. Bº. del Tutor:

Vº. Bº. del Tutor:

D. David García García

D. Cristina Arlanzón Pérez

Resumen

Uno de los principales desafíos en la gestión de la Unidad de Reanimación y Cuidados Críticos Postquirúrgicos del Servicio de Anestesiología y Reanimación (*URCCPQ*) del Hospital Universitario de Burgos (*HUBU*) radicaba en la evaluación periódica de sus actuaciones clínicas. Hasta la fecha, la jefatura de la unidad realizaba anualmente un análisis transversal de los pacientes, calculando los indicadores de forma estrictamente manual.

Aunque el muestreo transversal de los más de mil seiscientos pacientes anuales que recibe el servicio resultaba la única solución viable por su rapidez y bajo coste operativo, este enfoque metodológico presenta limitaciones intrínsecas: se restringe a descripciones estáticas y correlaciones puntuales, imposibilitando el análisis de la secuencia temporal y la evolución longitudinal de los eventos.

Para dar respuesta a esta limitación metodológica, este proyecto presenta el desarrollo de una herramienta/aplicación de software diseñada para el procesamiento y análisis automatizado de los datos clínicos. La aplicación es capaz de calcular veintiún indicadores de calidad en rangos configurables de uno a diez años. Además, incorpora un motor estadístico que evalúa la evolución del servicio mediante el análisis de tendencias, el cálculo del error típico interanual y la obtención del coeficiente de determinación (R^2) para validar la adherencia a modelos lineales.

La implementación de esta herramienta supone un salto cualitativo en la auditoría clínica de la unidad. Lo que previamente constituía un procedimiento estadístico laborioso y susceptible a errores humanos, se ha transformado en un flujo de trabajo automatizado. Ahora, la dirección médica puede obtener una evaluación integral e histórica del servicio con un solo *clic*, facilitando la toma de decisiones basada en datos y optimizando significativamente la gestión de los recursos en la *URCCPQ*.

Descriptores

Indicadores de calidad, *URCCPQ*, *HUBU*, aplicación web, software, Semicyuc, innovación tecnológica, bases de datos.

Abstract

One of the main challenges in the management of the Post-Surgical Resuscitation and Critical Care Unit of the Anesthesiology and Resuscitation Service (*URCCPQ*) at the University Hospital of Burgos (*HUBU*) lay in the periodic evaluation of its clinical performance. To date, the unit's management conducted an annual cross-sectional analysis of the patients, calculating the indicators in a strictly manual manner.

Although the cross-sectional sampling of the more than 1,600 patients the service receives annually was the only viable solution due to its speed and low operational cost, this methodological approach presents intrinsic limitations: it is restricted to static descriptions and point correlations, precluding the analysis of the temporal sequence and the longitudinal evolution of events.

To address this methodological limitation, this project presents the development of a software application designed for the automated processing and analysis of clinical data. The application is capable of calculating twenty-one quality indicators within configurable ranges from one to ten years. Furthermore, it incorporates a statistical engine that evaluates the service's evolution through trend analysis, the calculation of the interannual standard error, and the determination of the coefficient of determination (R^2) to validate adherence to linear models.

The implementation of this tool represents a qualitative leap in the unit's clinical auditing. What previously constituted a laborious statistical procedure susceptible to human error has been transformed into an automated workflow. Now, the medical direction can obtain a comprehensive and historical evaluation of the service with a single *click*, facilitating data-driven decision-making and significantly optimizing resource management in the *URCCPQ*.

Keywords

Quality indicators, *URCCPQ*, *HUBU*, web application, software, SEMICYUC, technological innovation, databases.

Índice general

Índice general	iii
Índice de figuras	iv
Índice de tablas	v
Introducción	1
1.1. Estructura de la Memoria	3
Objetivos	5
2.1. Objetivos Generales	5
2.2. Objetivos Específicos: Integración y Seguridad del Sistema .	6
2.3. Objetivos Específicos: Desarrollo Software	6
2.4. Objetivos Específicos: Desarrollo Hardware	7
Conceptos teóricos	9
3.1. SEMICYUC	9
3.2. ICCA	10
3.3. Desarrollo web	11
3.4. Arquitectura <i>Raspberry Pi</i> y Dispositivo Físico	12
3.5. El <i>Framework</i> Reflex y su Arquitectura de Comunicación . .	12
3.6. Despliegue automatizado: Contenedores <i>Docker</i> y <i>scripts</i> de automatización (.sh)	14
3.7. Gestión de Datos Persistentes: Bases de Datos Relacionales y <i>PostgreSQL</i>	15
3.8. Análisis Estadístico	16
3.9. Estado del arte y trabajos relacionados	17

Metodología	21
4.1. Desarrollo del proyecto	21
4.2. Descripción de los datos.	23
4.3. Metodología de Ingeniería del Software	25
4.4. Técnicas y herramientas.	29
Resultados	37
5.1. Resumen de resultados	37
Discusión	43
6.1. Validación y Explicación Matemática	44
6.2. Impacto de la Herramienta	46
6.3. Desafíos Técnicos y Evolución Arquitectónica	47
Conclusiones	51
Lineas de trabajo futuras	53

Índice de figuras

4.1. Desarrollo interactivo. <i>Elaboración propia</i>	28
4.2. Programas/Lenguajes usados. <i>Elaboración propia</i>	30
5.1. Módulos principales de carga/extracción de datos.	38
5.2. Gráfico de barras calculado por la aplicación.	39
5.3. Panel de la <i>BBDD</i>	39
5.4. Combinación de 3 indicadores.	40
5.5. Resumen de los indicadores.	41

Índice de tablas

4.1. Diccionario de variables (Parte 1): Datos generales, respiratorios y hemodinámica.	22
4.2. Diccionario de variables (Parte 2): Infectología, sedación, nutrición y seguridad.	23
4.3. Desglose de versiones del ecosistema <i>Frontend</i> autogenerado por <i>Reflex</i>	34
4.4. Desglose de versiones del ecosistema <i>Backend</i> (Entorno <i>Python</i>).	35

Introducción

Uno de los principales desafíos en la gestión de la Unidad de Reanimación y Cuidados Críticos Postquirúrgicos del Servicio de Anestesiología y Reanimación (*URCCPQ*) del Hospital Universitario de Burgos (*HUBU*) radica en la evaluación periódica de sus actuaciones clínicas. Hasta la fecha, la Dra. Cristina Arlanzón Pérez, jefa de la unidad y tutora clínica de este proyecto, realizaba anualmente un análisis transversal de los pacientes, calculando los indicadores de forma estrictamente manual.

Debido al alto volumen de pacientes que recibe la *URCCPQ* (cerca de mil seiscientos ingresos anuales), resultaba operativamente inviable para una única persona recolectar, procesar y calcular todos los indicadores exigidos por la Sociedad Española de Medicina Intensiva, Crítica y Unidades Coronarias (*SEMICYUC*) [?] necesarios para la auditoría de la unidad. Por consiguiente, se recurría a un muestreo transversal, siendo esta la única solución viable por su rapidez y bajo coste operativo. No obstante, este enfoque metodológico presenta limitaciones intrínsecas: se restringe a descripciones estáticas y correlaciones puntuales, imposibilitando el análisis de la secuencia temporal y la evolución longitudinal de los eventos clínicos.

Ante estas limitaciones metodológicas, y motivada por la urgencia de una inminente auditoría para la certificación del punto 9.3.1 de la norma *ISO 9001/2015* [?] en el mes de marzo, la dirección médica solicitó la colaboración de la Universidad de Burgos (*UBU*). El objetivo inicial fue incorporar a un estudiante en prácticas del grado de Ingeniería de la Salud para desarrollar una solución informática capaz de solventar estas carencias analíticas.

El resultado de esa fase inicial fue un programa robusto construido íntegramente en *Python*. Este *script*, ejecutado mediante consola, contaba con un núcleo de procesamiento y cálculo de datos preciso, funcional y debidamente

documentado. Sin embargo, carecía de un acabado profesional orientado al usuario final. Su manejo requería conocimientos técnicos específicos, lo cual suponía una barrera de entrada para el personal clínico, y su salida de datos se limitaba a la generación de archivos *CSV* y resultados aislados por consola.

El presente Trabajo Fin de Grado (*TFG*) tiene como motivación principal expandir y profesionalizar esa base algorítmica inicial, desarrollando una solución tecnológica integral que combina *hardware* y *software*. El objetivo es generar una aplicación web profesional, dotada de una interfaz clínica intuitiva y lista para ser utilizada por cualquier profesional sanitario, independientemente de su nivel de conocimientos informáticos, desarrollada a través de *Reflex* un *framework*¹ de código abierto para *Python* que permite crear aplicaciones *web full-stack*². Esta plataforma *web full-stack* garantiza la seguridad y persistencia de la información mediante una base de datos *PostgreSQL* gestionada a través de *SQLModel*, lo que permite una administración robusta de usuarios y registros de auditoría. El despliegue de la aplicación se realizará en la *intranet* del Hospital Universitario de Burgos utilizando la tecnología de contenedores *Docker*. Esta transición hacia un entorno virtualizado y centralizado no solo optimiza la escalabilidad del sistema, sino que refuerza el cumplimiento de la *LOPD* [?] al operar dentro de la infraestructura tecnológica protegida del propio centro hospitalario.

La herramienta es capaz de calcular veintiún indicadores de calidad en rangos configurables de uno a diez años e incorpora un motor estadístico que evalúa la evolución del servicio mediante el análisis de tendencias, el cálculo del error típico interanual y la validación de la adherencia a modelos lineales mediante el coeficiente de determinación (R^2).

Adicionalmente, el proyecto ha expandido la accesibilidad de la herramienta en dos vertientes:

- Despliegue automatizado mediante contenedores *Docker* en la *intranet* del *HUBU*, integrándose directamente con su servidor *PostgreSQL* para la persistencia de la base de datos.
- Desarrollo paralelo de un terminal físico portátil (denominado internamente “*tablet médica*”), constituido por una arquitectura *Raspberry Pi* y una pantalla táctil, que dota al personal clínico de movilidad para utilizar la aplicación fuera del hospital.

¹Conjuntos de herramientas y reglas que agilizan el desarrollo de software

²*frontend* y *backend*

Cabe señalar que la numeración total alcanza las 54 páginas. Sin embargo, 4 de ellas corresponden a separadores de sección u hojas en blanco introducidas para facilitar la legibilidad del lector. Por tanto, la extensión real de contenido técnico se sitúa en 50 páginas, habiéndose realizado un importante esfuerzo de síntesis para ajustarse lo máximo posible al límite de páginas sin comprometer la claridad de la exposición.

1.1. Estructura de la Memoria

El presente documento detalla el proceso integral de diseño y desarrollo del sistema. Para facilitar su lectura, la memoria se ha estructurado en los siguientes capítulos:

- **Capítulo 2. Objetivos:** Define el propósito principal del *TFG* y los objetivos específicos técnicos y funcionales a alcanzar.
- **Capítulo 3. Fundamentos Teóricos y Estado del Arte:** Proporciona el marco clínico y tecnológico necesario para comprender el proyecto, incluyendo un análisis de soluciones alternativas existentes.
- **Capítulo 4. Metodología:** Detalla las herramientas de *software* y *hardware* empleadas, así como el flujo de trabajo seguido durante el ciclo de vida del desarrollo.
- **Capítulo 5. Resultado e Implementación:** Relata los resultados obtenidos del proyecto.
- **Capítulo 6. Discusión:** Evalúa la efectividad del producto final frente a los objetivos iniciales y su impacto en la gestión de la unidad.
- **Capítulo 7. Conclusiones:** Sintetiza los logros del proyecto y detalla algunos de los aspectos más relevantes del proyecto.
- **Capítulo 8. Líneas Futuras:** Propone posibles líneas de expansión y mejora.

Finalmente, el documento incluye una sección de **Anexos** (Apéndices A-I) que complementa la memoria con documentación técnica, manuales de usuario y programador, planes de pruebas, un análisis de sostenibilidad y la trazabilidad de las herramientas utilizadas durante el desarrollo.

Objetivos

Este proyecto busca proporcionar una herramienta para la mejora del cálculo de indicadores clínicos mediante la aplicación de tecnologías avanzadas de análisis de datos. La idea surge de la necesidad de ampliar un trabajo previo realizado en las prácticas curriculares, superando las limitaciones metodológicas del cálculo manual.

A continuación, se enumeran y explican los objetivos generales y específicos que guían la realización de este proyecto.

2.1. Objetivos Generales

El proyecto se fundamenta en un único objetivo principal, del cual derivan las diferentes ramas de desarrollo técnico necesarias para su implantación en la unidad de Reanimación y Cuidados Críticos postquirúrgicos del Servicio de Anestesiología y Reanimación de Burgos (*HUBU*).

- **Desarrollar y validar** una plataforma *web* clínica que automatice el análisis del desempeño de la unidad y la generación de informes estadísticos, con el fin de reducir drásticamente el tiempo de gestión administrativa y mejorar la eficiencia de los flujos de trabajo del personal médico [?].

2.2. Objetivos Específicos: Integración y Seguridad del Sistema

Para alcanzar el objetivo general cumpliendo con los estrictos estándares de privacidad hospitalaria, se establecen los siguientes objetivos secundarios:

1. **Desplegar** el sistema de forma local y centralizada a través de la *intranet* del hospital. Se debe prescindir de servicios externos como *Reflex Cloud* para imposibilitar la salida de datos de los pacientes hacia servidores con *IP* pública.
2. **Establecer** un flujo de recolección semiautomática y seudonimizada de variables clínicas, almacenando los registros de forma persistente en la base de datos *PostgreSQL* interna del hospital para su posterior análisis y auditoría.

2.3. Objetivos Específicos: Desarrollo Software

Para materializar el motor analítico de la herramienta, se plantean las siguientes metas a nivel de código y diseño:

1. **Procesar** archivos *CSV* locales, implementando un módulo seguro de carga, lectura y posterior eliminación que garantice el control total sobre la persistencia en el equipo.
2. **Extraer y analizar** registros de la *BBDD* mediante arquitectura *Zero-Disk*³. Mediante *SQLModel*, los datos se inyectarán en *DataFrames* de *Pandas* a través de *buffers*⁴ temporales. Esto asegura el procesamiento a velocidad nativa y la destrucción de la información mediante el *Garbage Collector*⁵ al cerrar la sesión, en estricto cumplimiento de la *LOPD*.

³Arquitectura o metodología de desarrollo donde las aplicaciones y sistemas operativos están diseñados para funcionar sin depender de un disco duro local (*HDD* o *SSD*) para el almacenamiento de datos persistentes o la ejecución de archivos del sistema durante su operación.

⁴Espacio de almacenamiento efímero en la memoria *RAM* que retiene datos durante su transferencia entre procesos, evitando el uso del disco duro.

⁵Mecanismo automático de gestión de memoria que elimina objetos que ya no se utilizan en el programa, liberando espacio en la memoria *RAM*.

3. **Diseñar** una interfaz intuitiva, accesible y *Zero-Install* que ofrezca una experiencia de usuario satisfactoria a todo el personal clínico, independientemente de su nivel de competencia tecnológica.

2.4. Objetivos Específicos: Desarrollo Hardware

Finalmente, para dotar al proyecto de versatilidad en escenarios fuera de la *intranet* hospitalaria, se plantea el siguiente objetivo físico:

1. **Construir y optimizar** un terminal físico autónomo y portátil (arquitectura *Raspberry Pi*, denominada internamente “*tablet médica*”). Este dispositivo dotará al facultativo de total movilidad para utilizar la aplicación y descargar registros estadísticos anonimizados en formato *CSV* sin comprometer la seguridad del servidor central.

Conceptos teóricos

3.1. SEMICYUC

Para comprender la magnitud e importancia de la evaluación clínica en la *URCCPQ*, es imprescindible fundamentar las métricas utilizadas en estándares científicos reconocidos. En el ámbito nacional, el organismo de referencia es la Sociedad Española de Medicina Intensiva, Crítica y Unidades Coronarias (*SEMICYUC*).

La *SEMICYUC* tiene como objetivo prioritario la mejora continua de la calidad asistencial y la seguridad del paciente crítico. Para ello, la sociedad desarrolla y actualiza periódicamente el manual de “Indicadores de Calidad en el Enfermo Crítico” [?]. Este documento define un conjunto de variables objetivas y cuantificables que abarcan todas las dimensiones de la asistencia como la estructura, los procesos y sus resultados. Cada indicador incluye su justificación clínica, la fórmula matemática exacta para su cálculo, la población de exclusión e inclusión, y el estándar de calidad (el umbral porcentual que se considera aceptable o de excelencia).

Relevancia en el desarrollo del proyecto

La integración de las directrices de la *SEMICYUC* constituye el núcleo funcional y la justificación clínica de la aplicación de *software* desarrollada en este *TFG*. La relevancia de este organismo en el proyecto se articula en torno a los siguientes puntos:

- **Estandarización y Rigor Científico:** Los veintiún indicadores calculados por la herramienta no son métricas arbitrarias, sino que responden estrictamente a las fórmulas y criterios de inclusión dictados

por la *SEMICYUC*. Esto garantiza que los resultados obtenidos por el *software* tengan validez médica real.

- **Certificación y Auditoría:** La evaluación periódica de estos indicadores es un requisito indispensable para que el Hospital Universitario de Burgos (*HUBU*) pueda someterse a auditorías externas y mantener certificaciones de calidad internacionales, como la norma *ISO*. La automatización de este proceso elimina el sesgo del cálculo manual y asegura la trazabilidad de los datos.
- **Toma de Decisiones Basada en la Evidencia:** Al sistematizar el cálculo de los indicadores de la *SEMICYUC*, la dirección médica de la unidad dispone de un cuadro de mando objetivo que permite detectar desviaciones de seguridad, evaluar la eficacia de los protocolos de actuación y optimizar los recursos hospitalarios basándose en evidencias tangibles.

3.2. *ICCA*

En la Unidad de Reanimación y Cuidados Críticos postquirúrgicos del Servicio de Anestesiología y Reanimación (URCCPQ) del Hospital Universitario de Burgos (*HUBU*) se emplea el software de gestión de datos clínicos de Philips, denominado *IntelliSpace Critical Care and Anesthesia (ICCA)*.

Este sistema avanzado, diseñado específicamente para entornos de cuidados intensivos, busca aportar a los profesionales sanitarios un rápido acceso a datos completos y actualizados mediante la organización y centralización de la información del paciente (documentos de admisión, signos vitales, notas de consulta, resultados de laboratorio...), permitiendo desarrollar y almacenar historias clínicas, evolutivos, así como pautar tratamientos.

Ventajas del *ICCA*

El *ICCA*, por tanto, ofrece un apoyo en la toma de decisiones clínicas, al permitir la transformación de los datos en información procesable y proporcionar una documentación estructurada y estandarizada [?].

Además, múltiples dispositivos (sistemas de monitorización, respiradores...) y otros sistemas de información hospitalaria (laboratorio, farmacia...) se integran con el *ICCA* lo que facilita enormemente la obtención de datos y su sincronización con toda la información del paciente, permitiendo lograr una monitorización de éste mucho más eficiente y de mejor calidad, así como

generar un repositorio de datos para desarrollar proyectos de investigación [?].

Limitaciones de los datos

No obstante, existen grandes dificultades en el análisis de esos datos, ya que el software no ofrece ningún tipo de herramienta para el análisis profundo, más allá de gráficos individuales.

Actualmente, se están implementado herramientas de indicadores que facilitan el cálculo de estos. Dichas herramientas utilizan técnicas avanzadas para procesar los datos médicos y extraen conceptos de una forma eficiente; sin embargo, desafortunadamente estas herramientas no cumplían con las necesidades específicas de la unidad.

Integración del *ICCA* en el proyecto

La unidad lleva utilizando el *ICCA* desde el año 2024, el cual a lo largo de estos años se ha actualizado en varias ocasiones para disponer de la última versión del programa.

Este proyecto aborda una necesidad existente en el cálculo de indicadores de calidad en el enfermo crítico, aportando una solución de ingeniería informática que mejore la calidad y eficiencia en la gestión y procesamiento de la información clínica, facilitando la generación de un informe del desempeño de la unidad a lo largo del año.

3.3. Desarrollo web

Para el proceso de desarrollo de una aplicación web, o cualquier otro proyecto software, es imprescindible conocer y diferenciar los conceptos de *front-end* y *back-end*. Cada uno de estos ámbitos emplea diferentes lenguajes de programación, y una conexión e interacción eficaz entre ambos es esencial para obtener un software funcional [?].

- **Front-end:** Se refiere a la codificación o diseño de la interfaz de usuario. Este aspecto considera elementos como la accesibilidad, estructura de navegación, capacidad de respuesta y animaciones. Utiliza lenguajes como *CSS*, *HTML* y *JavaScript*, pero también puede incluir *frameworks* como *Bootstrap*. [?] [?]

- **Back-end:** Este campo está centrado en la funcionalidad, se programa con el objetivo de lograr eficazmente las acciones disponibles para el usuario desde el *front-end*. Gestiona los recursos, implementa la lógica del sistema e integra componentes como servidores *web*, *APIs* y bases de datos. Entre los lenguajes de programación empleados están *Java* y *Python*. [?] [?]

3.4. Arquitectura *Raspberry Pi* y Dispositivo Físico

Para dotar al personal médico de movilidad y facilitar el uso de la herramienta analítica fuera de sus áreas de trabajo, se ha desarrollado un terminal clínico portátil basado en la arquitectura *Raspberry Pi*.

Fundamentos de la Arquitectura *SBC* y *ARM*

Una *Raspberry Pi* es un ordenador de placa reducida o *SBC* (*Single Board Computer*). [?] A diferencia de los ordenadores tradicionales, que distribuyen sus componentes en una placa base de gran tamaño, un *SBC* integra todos los elementos esenciales (procesador, memoria *RAM*, controladores de entrada/salida y conectividad de red) en una única placa de circuito impreso de dimensiones mínimas.

El núcleo de esta arquitectura se basa en un *SoC* (*System on a Chip*), que agrupa la Unidad Central de Procesamiento (*CPU*) y la Unidad de Procesamiento Gráfico (*GPU*) en un solo circuito integrado. Estos procesadores utilizan la arquitectura *ARM* (*Advanced RISC Machine*), caracterizada por implementar un conjunto reducido de instrucciones (*RISC*). Esta eficiencia computacional permite obtener un alto rendimiento con un consumo energético drásticamente inferior al de los procesadores convencionales, generando a su vez una mínima disipación térmica. [?]

3.5. El *Framework* *Reflex* y su Arquitectura de Comunicación

Para el desarrollo de la aplicación web de este proyecto, se ha empleado *Reflex*, un *framework web full-stack* de código abierto que permite construir aplicaciones web interactivas utilizando íntegramente el lenguaje *Python*. La principal innovación de esta herramienta radica en su capacidad para

abstraer la complejidad de programar la interfaz visual (*frontend*) y la lógica del servidor (*backend*) por separado. [?]

Durante el proceso de compilación y ejecución, *Reflex* divide el sistema en dos entornos independientes pero altamente coordinados:

- **Frontend:** Una interfaz de usuario moderna y reactiva basada en *React* y *Next.js*.
- **Backend:** Un servidor de alto rendimiento basado en *FastAPI* que aloja la lógica de negocio⁶ y los cálculos matemáticos.

Para que estos dos entornos se comuniquen de manera eficiente y proporcionen una experiencia de usuario fluida, la arquitectura subyacente de *Reflex* hace uso de dos protocolos de comunicación de red fundamentales: la *API REST* y los *WebSockets*.

Es fundamental comprender que *FastAPI* no es excluyente con el uso de *WebSockets*, sino que actúa como el motor principal que gestiona ambos protocolos simultáneamente.

Mientras que el protocolo *WebSocket* se reserva de manera exclusiva para mantener la sincronización del estado (*State Management*) y la interactividad en tiempo real de la interfaz gráfica, *Reflex* sigue utilizando el estándar *API REST* provisto por *FastAPI* para operaciones estructurales.

El modelo de petición-respuesta: *API REST*

REST (*Representational State Transfer*) es un protocolo de comunicación estándar basado en un modelo cliente-servidor sin estado (*stateless*). En este paradigma, el cliente (el navegador *web*) envía una petición *HTTP* al servidor solicitando una acción o un recurso. El servidor procesa la solicitud, devuelve la respuesta correspondiente y, acto seguido, finaliza la conexión.

En el contexto de una interfaz clínica moderna, depender exclusivamente de *REST* presentaría limitaciones operativas. Por ejemplo, cada vez que el personal médico seleccionase un gráfico diferente o solicitase el cálculo de un indicador, el navegador se vería forzado a recargar la página *web* por completo para mostrar la gráfica actualizada. No obstante, dado que el *backend* de *Reflex* está construido sobre *FastAPI*, la aplicación dispone de la capacidad nativa de exponer rutas *REST* tradicionales. Esto resulta de gran

⁶Conjunto de reglas, algoritmos y procesos de una organización que definen cómo se crean, almacenan y transforman los datos en un sistema informático

utilidad para operaciones estáticas, como la carga inicial masiva de bases de datos de pacientes, o para permitir futuras integraciones automatizadas con los sistemas de información del *HUBU*. [?]

Comunicación en tiempo real: *WebSockets*

Para superar las limitaciones del modelo tradicional y dotar a la herramienta de interactividad en tiempo real, *Reflex* fundamenta su gestión de estado (*State Management*) en el protocolo *WebSockets*. A diferencia de *REST*, un *WebSocket* establece un canal de comunicación bidireccional y persistente entre el navegador y el servidor. Una vez establecida la conexión, esta permanece abierta, permitiendo a ambas partes enviar y recibir información de forma asíncrona en cualquier momento sin necesidad de reabrir el canal constantemente.

Esta tecnología es el verdadero motor dinámico de la aplicación desarrollada para la *URCCPQ*. Esta arquitectura garantiza que el personal médico disponga de una herramienta de auditoría ágil y robusta, capaz de manejar grandes volúmenes de datos históricos sin comprometer el rendimiento visual de la aplicación. [?]

3.6. Despliegue automatizado: Contenedores *Docker* y *scripts* de automatización (*.sh*)

Para garantizar que la herramienta sea accesible en el *HUBU* por personal sin formación técnica, el despliegue se centraliza en la *intranet* hospitalaria mediante contenedores *Docker* [?]. Esta solución permite integrar la aplicación con el servidor *PostgreSQL* institucional, asegurando un entorno robusto y el cumplimiento estricto de la *LOPD*.

Docker es una tecnología de virtualización de nivel de sistema operativo que empaqueta el *software* y sus dependencias en unidades estandarizadas. Para el terminal portátil basado en *Raspberry Pi*, se ha desarrollado un *script* de automatización (*.sh*), el cual se ejecuta automáticamente en el arranque del dispositivo. Permitiendo el análisis de los datos tras su extracción anónima a través de la aplicación.

Esta arquitectura de ingeniería elimina la barrera tecnológica para el facultativo, transformando un sistema de alta seguridad y procesamiento

en memoria en una solución *plug and play* perfectamente integrada en la infraestructura del *HUBU*.

3.7. Gestión de Datos Persistentes: Bases de Datos Relacionales y *PostgreSQL*

Para complementar la arquitectura de procesamiento clínico en memoria (*Zero-Disk*) y cumplir estrictamente con los requisitos de trazabilidad de la *LOPD*, el sistema requiere un almacenamiento persistente y seguro para el control de accesos. Para ello, se ha implementado una base de datos relacional. [?]

Una base de datos relacional organiza la información en tablas bidimensionales (filas y columnas) vinculadas entre sí mediante identificadores únicos, garantizando la consistencia e integridad de la información. Para interactuar con esta arquitectura se emplea *SQL* (*Structured Query Language*) [?], el lenguaje de programación estándar diseñado para administrar y consultar este tipo de sistemas.

En el contexto de este *TFG*, se ha desplegado *PostgreSQL* en los servidores del *HUBU*. Se trata de un potente Sistema de Gestión de Bases de Datos Relacionales (*RDBMS*) de código abierto, reconocido por su robustez empresarial y sus avanzados protocolos de seguridad. Para la gestión de la base de datos, el sistema emplea una arquitectura estructurada de forma modular. A nivel de abstracción lógica, se utiliza *SQLModel* como *ORM*⁷ (*Object-Relational Mapper*), lo que permite interactuar con los datos mediante paradigmas orientados a objetos en *Python*, agilizando el desarrollo y previniendo inyecciones *SQL*. El control de versiones y la evolución del esquema estructural recaen sobre *Alembic*, la herramienta encargada de traducir cualquier modificación en los modelos a sentencias *DDL*⁸ (*Data Definition Language*) para actualizar las tablas de forma segura e incremental sin pérdida de información. Finalmente, a nivel de red, la comunicación física con el motor *PostgreSQL* y la ejecución de todas estas transacciones a bajo nivel se delegan en *psycopg2*, el adaptador de bases de datos estándar y de alto rendimiento. La base de datos a parte de almacenar los registros de los pacientes cumple dos funciones críticas:

⁷Técnica y herramienta de *software* en informática que actúa como un puente entre la programación orientada a objetos (clases, objetos) y las bases de datos relacionales (tablas, filas, columnas).

⁸Conjunto de comandos en informática utilizados para definir, modificar y eliminar la estructura de los objetos dentro de una base de datos.

1. **Autenticación Segura (UsuarioDb):** Almacena los perfiles de los facultativos sin guardar credenciales en texto plano. Utiliza funciones de derivación de claves criptográficas (*hashing*) unidireccionales mediante *Bcrypt*, blindando el sistema contra la ingeniería inversa.
2. **Auditoría y Trazabilidad (Auditoria):** Registra de forma silenciosa e inalterable cada acción crítica realizada en la plataforma (inicios de sesión, extracción o análisis de datos). Este registro vincula la identidad del médico con una marca de tiempo exacta, proporcionando la trazabilidad exigida en el ámbito sanitario.

Esta infraestructura permite que los registros de los pacientes, el control de acceso y la auditoría legal quedan respaldados de forma permanente y segura en la base de datos *PostgreSQL*. Sin embargo, fiel a la arquitectura *Zero-Disk*, el sistema asegura que estos datos se mantengan y procesen exclusivamente en la memoria *RAM* de la aplicación durante el tiempo de ejecución. De esta manera, el almacenamiento persistente en la base de datos sirve como respaldo robusto, mientras que la volatilidad en la *RAM* garantiza que no queden rastros físicos de información sensible en el servidor al finalizar la sesión.

3.8. Análisis Estadístico

El motor estadístico integrado en la herramienta permite trascender la mera descripción estática de los datos, aportando un enfoque predictivo y de control de calidad continuo mediante tres pilares fundamentales:

- **Análisis de tendencias:** Evalúa la evolución histórica de los indicadores clínicos a lo largo del tiempo, identificando patrones direccionales (incrementos o decrementos) que permiten a la dirección médica anticipar escenarios y evaluar el impacto a largo plazo de las nuevas guías clínicas. [?]
- **Cálculo del error típico interanual:** Cuantifica la dispersión estadística de los datos reales respecto a la línea de tendencia central. Proporciona a los facultativos una medida objetiva de la estabilidad de la unidad, indicando cuánto fluctúa habitualmente una métrica de un año a otro. [?]
- **Coefficiente de determinación (R^2):** Actúa como validador matemático del modelo predictivo. Este parámetro, acotado entre 0 y 1, mide

la proporción de la varianza explicada por el modelo; un valor de R^2 cercano a 1 confirma que la evolución del indicador se adhiere fielmente a una progresión lineal, garantizando que las tendencias observadas son estadísticamente significativas y no fruto de la aleatoriedad. [?]

3.9. Estado del arte y trabajos relacionados

El termino estado del arte (*state of the art*) hace referencia a una **revisión sistemática** que recopila, analiza y sintetiza el conocimiento y resultados mas recientes sobre un tema especifico en un momento dado. Permite que los investigadores adopten una postura más crítica sobre los aspectos limitantes o problemáticas ya resueltas de un tema.

Por lo que en esta sección se comentaran todas aquellas herramientas que existen, que existirán y sus posibles implementaciones.

Estado actual

Si bien existen diversas herramientas de análisis clínico en la actualidad, no se dispone de una solución comercial o de código abierto lo suficientemente adaptada a los indicadores *SEMICYUC* y accesible para su despliegue directo en la unidad.

Actualmente, el cálculo de estos indicadores en las *UCIs* españolas se resuelve utilizando tres grandes grupos de herramientas.

1. **Módulos de informes de los propios fabricantes (*Philips IC-CA*):** El propio programa usado en la unidad (*IntelliSpace Critical Care and Anesthesia - ICCA* de *Philips*) tiene un módulo llamado **DAR** (*Data Analysis and Reporting*).

- a) Este es un *software* privativo, que está integrado directamente en la base de datos del hospital.
- b) Presenta una gran rigidez debido al uso de plantillas genéricas internacionales. Adaptarlas para que calculen específicamente las fórmulas de la Sociedad Española (*SEMICYUC*) requiere contratar técnicos especialistas de *Philips*, lo cual necesita una gran inversion de tiempo y dinero.[?]
- c) A estos inconvenientes se suma su baja portabilidad ya que no es una aplicación ligera, la cual un médico pueda ejecutar en

cualquier ordenador para cargar archivos; es un sistema pesado y de acceso restringido.

2. **Herramientas de Business Intelligence (*Power BI*, *Tableau*, *QlikView*):** Muchos hospitales modernos exportan los datos a un *Data Warehouse* y usan herramientas de *BI* como *Micorsoft Power BI* para la realización de cuadros de mando

- a) Son herramientas gráficas ultra potentes y multipropósito
- b) *Power BI* es un lienzo en blanco. El personal clínico tendría que aprender a programar en lenguaje *DAX* o *SQL* para limpiar los nulos, parsear las fechas y escribir las fórmulas matemáticas requeridas.[?]
- c) La aplicación desarrollada en *Python* limpia automáticamente los “datos sucios” (los típicos errores al exportar archivos, palabras excluidas, etc...). *Power BI* sufre mucho si los datos de origen cambia de formato o viene mal tabulado. El *backend* desarrollado en este proyecto está hecho a medida para sobrevivir a estos errores.

3. **Plataformas de Registros Nacionales (*REDCap*, *ENVIN*, *RETRAUCI*):** Existen multitud de plataformas web institucionales (algunas de la propia *SEMICYUC*) donde los médicos introducen datos para calcular ciertos riesgos (como la herramienta *RETRASCORE* para traumas) o llevar registros de infecciones.

- a) Son bases de datos académicas estandarizadas a nivel nacional.
- b) El mayor inconveniente de estas plataformas radica en la **obligación al facultativo de introducir los datos manualmente** paciente por paciente. La herramienta propuesta elimina esto por completo puesto que procesa los registros desde la base de datos de manera inmediata, automatizando meses de trabajo en segundos.[?]

Resumiendo, en internet hay calculadoras médicas simples y hay gigantescos sistemas de datos de hospital. Este *software* es el **punto perfecto** entre ambos mundos: tiene la potencia matemática para procesar *big data*, pero la interfaz sencilla de una web orientada al usuario final.

Uso de la IA

Actualmente la Inteligencia artificial *IA* esta tomando un papel de suma importancia en el desarrollo de software clínico. La inteligencia artificial puede ser un grandísimo compañero y ayudante a la hora de implementar estas nuevas necesidades; sin embargo su uso indiscriminado puede conllevar graves problemas:

1. **Plagio de código humano:** Como ya hemos comentado anteriormente, existen algunas herramientas que cumplen un rol parecido al cubierto por este proyecto, por lo que el uso de la *IA* en alguna solución parecida (como la aquí presente) podría suponer un grave riesgo de plagio hacia otros compañeros del sector.
2. **Calidad del Código:** A pesar de que el uso de la *IA* pueda servir de gran ayuda, las evidencias delatan que el simple uso de copiar y pegar el código, todavía no es una gran idea. Varios estudios [?] aclaran que aunque sus resultados cada vez son mas profesionales y rigurosas, las *IAs* generativas están lejos de los resultados óptimos; necesitando de una persona profesional que se encargue de corregir cierto errores y comprobando la calidad del código entregado.

Por lo que el uso a ciegas de estas herramientas no es precisamente útil. Precisan de una persona competente que sepa estructurar y dirigir el proyecto.

3. **Privacidad de los datos:** La privacidad de los datos en el ámbito clínico es un aspecto muy importante a tener en cuenta. Este mismo proyecto se ha visto limitado en diferentes ocasiones debido al tratamiento de datos sensibles.

Este tipo de herramientas utilizan todos los datos y consultas que hacemos con ellos para desarrollarse y aprender. Por lo que en un ámbito sanitario en el que nos encontramos, tenemos que tener claro que tipo de consultas e información le facilitamos a la *IA*.

En definitiva, la Inteligencia Artificial en la ingeniería de la salud no debe entenderse como un sustituto del desarrollador, sino como un copiloto. Su aprovechamiento real exige un uso ético, un filtrado estricto de la información que se le proporciona y una validación humana constante para garantizar la seguridad técnica, clínica y legal del proyecto.

Metodología

4.1. Desarrollo del proyecto

El desarrollo del proyecto consta de varias fases de colaboración entre el servicio de la *URCCPQ* y el Grado de Ingeniería de la Salud de la *UBU*, las cuales a continuación se detallaran:

- **1ª Fase:** Seudonimización por parte de los clínicos. Datos como la presión arterial media, los días en ventilación mecánica o el uso de corticoides. Todos estos datos se recogerán e incluirán en la base de datos de forma pseudonimizada. Para facilitar su visualización y comprensión, el diccionario de variables se ha fragmentado en dos bloques lógicos. La Tabla 4.1 detalla los datos de ingreso, soporte respiratorio y monitorización hemodinámica; mientras que la Tabla 4.2 recoge los parámetros de infectología, sedación, nutrición y seguridad. Se tendrán en cuenta los siguientes criterios de inclusión/exclusión:
 - **Inclusión:** Se recogen los datos necesarios de todos los pacientes que hayan estado ingresados en el servicio de la *URCCPQ*.
 - **Exclusión:** Pacientes que pasan al servicio de la *URCCPQ* debido a que el área de **Unidad de Recuperación Postanestésica (URPA)** permanece cerrada, perteneciendo originalmente a la *URPA*.
- **2ª Fase:** Desarrollo de la herramienta con datos incluidos de manera aleatoria.

Variable Clínica	Tipo de Dato	Descripción / Formato
A. Datos Generales y Fechas		
Fecha Ingreso	Fecha	Fecha del ingreso en <i>UCI</i> (<i>dd/mm/aaaa</i>).
Fecha Alta	Fecha	Fecha de salida de la <i>UCI</i> .
Reingreso 48	Booleano	<i>True</i> / <i>False</i> .
Especialidad de Ingreso	Texto	Área médica de procedencia (Ej: Cardiología).
Ingreso por Urgencia	Booleano	<i>True</i> / <i>False</i> .
Fallecimiento	Booleano	<i>True</i> / <i>False</i> .
APACHE	Entero	Puntuación de gravedad al ingreso.
B. Ventilación Mecánica y Respiratorio		
VMI	Booleano	Indica si el paciente requirió Ventilación Mecánica Invasiva.
Días VMI	Número	Días totales de soporte respiratorio.
Barotrauma	Booleano	<i>True</i> / <i>False</i> .
Grados Inclinación	Entero	Rango posicional de 0 a 90 grados.
Episodios NAV	Booleano	<i>True</i> / <i>False</i> .
Registro Intubaciones	Texto	Fechas separadas por comas (Ej: "14/10/2025, 18/10...").
Extubación Programada	Booleano	<i>True</i> / <i>False</i> .
Extubación por Maniobras	Booleano	Retirada accidental del soporte respiratorio.
C. Sepsis y Monitorización Hemodinámica		
Sepsis	Booleano	<i>True</i> / <i>False</i> .
Shock Séptico	Booleano	<i>True</i> / <i>False</i> .
PAM (Ingreso / 6h / 24h)	Entero	Presión Arterial Media (<i>mmHg</i>) en dichos periodos.
Diuresis (Ingreso / 6h / 24h)	Decimal	Volumen urinario (<i>ml/kg/h</i>) en dichos periodos.
Lactato (Ingreso / 6h / 24h)	Decimal	Nivel de ácido láctico (<i>mmol/L</i>) en dichos periodos.

Tabla 4.1: Diccionario de variables (Parte 1): Datos generales, respiratorios y hemodinámica.

- **3ª Fase:** Privacidad y Cumplimiento de la *LOPD*. La plataforma implementa medidas arquitectónicas estrictas para garantizar que los datos clínicos permanezcan aislados y sean efímeros:

1. **Aislamiento de Red:** El sistema opera de forma cautiva dentro de la *intranet* del *HUBU* a través de contenedores *Docker*. Al no requerir conexión a internet pública, se elimina el riesgo de exfiltración de la información hacia el exterior.
2. **Arquitectura *Zero-Disk*:** El sistema elude cualquier tipo de escritura de archivos temporales en el disco duro. Los registros clínicos se manejan exclusivamente a través de *buffers* en la memoria *RAM*.
3. **Destrucción Volátil:** Se ha superado la necesidad de utilizar *scripts* de limpieza secundarios o *watchdogs*. Al interrumpirse la conexión (*WebSocket*) por el cierre del navegador o al finalizar la sesión, el *Garbage Collector* de *Python* purga instantáneamente la memoria, asegurando que no quede ningún remanente físico o lógico de los datos.

- **4ª Fase:** Validación de la herramienta.

Variable Clínica	Tipo de Dato	Descripción / Formato
D. Infectología y Antibióticos		
Tratamiento adecuado	Booleano	<i>True / False.</i>
Cobertura empírica	Booleano	<i>True / False.</i>
Resistencia antibióticos	Booleano	<i>True / False.</i>
Número Episodios Bacteriemia	Númérico	Total de episodios sufridos.
Número total de días CVC	Númérico	Total de días con catéteres venosos centrales.
E. Sedación y Monitorización Neurológica		
RASS / RASS objetivo	Entero	Escala de agitación/sedación (-5 a +4) real y prescrita.
BIS / BIS objetivo	Entero	Índice bispectral (0 a 100) real y prescrito.
Ventana de sedación	Númérico	Días de interrupción diaria programada de sedación.
F. Nutrición y Riesgos Gastrointestinales		
Fecha Inicio NE	Fecha	Inicio de Nutrición Enteral (<i>dd/mm/aaaa</i>).
Omeprazol	Booleano	Recepción de profilaxis gástrica.
Tratamiento Corticoides	Booleano	<i>True / False.</i>
Antecedentes Hemorragia	Booleano	<i>True / False.</i>
Coagulopatía	Booleano	<i>True / False.</i>
Insuficiencia Renal	Booleano	<i>True / False.</i>
Insuficiencia Hepática	Booleano	<i>True / False.</i>
G. Seguridad y Otros Cuidados		
UPP	Booleano	Presencia constatada de Úlceras por Presión.
Profilaxis TVP	Booleano	<i>True / False.</i>
TVP	Booleano	Trombosis Venosa Profunda confirmada.
Glucemia	Númérico	Nivel de glucosa en sangre (<i>mg/dl</i>).
Tratamiento Insulina	Booleano	<i>True / False.</i>
Traslado Intrahospitalario	Booleano	<i>True / False.</i>
Listado verificación	Booleano	<i>Check-list</i> de seguridad cumplimentado en traslado.
Eventos Adversos Traslado	Booleano	<i>True / False.</i>
Actos Transfusionales	Booleano	<i>True / False.</i>
Sangrado Activo	Booleano	<i>True / False.</i>
Cantidad Transfusión	Entero	Número diario de concentrados de hematíes administrados.

Tabla 4.2: Diccionario de variables (Parte 2): Infectología, sedación, nutrición y seguridad.

4.2. Descripción de los datos.

Este proyecto se cimenta sobre los datos de miles de personas, extraídos de sus Historias Clínica del Sistema Nacional de Salud (*HCDSNS*) [?]. Estos datos ocupan el lugar mas elevado en cuanto a la protección y privacidad. Dichos datos pueden revelar ideología, afiliación sindical, religión, creencias, origen racial, haciendo que ante una filtración, se podría generar una discriminación laboral, problemas con seguros o daños psicológicos.

El programa anteriormente descrito (*ICCA*) tiene una clara falta de exportación de estos datos. Debido a la sensibilidad de los mismo, *ICCA* impide su utilización directa. El primer acceso a los datos que se propuso en el proyecto fue redactando un escrito a la dirección del centro, detallando y solicitando la concesión de los datos totalmente anonimizados. Tras la ausencia de respuesta por parte del centro, la jefatura de la unidad propuso una método de recolección semiautomático y seudonimización.

En lugar de emplear archivos de texto externos, la definición de las variables clínicas y sus tipos se ha integrado directamente en el formulario de adición de registros de la aplicación *Reflex*. Los facultativos generalmente médicos residentes⁹ introducen la información de forma seudonimizada a través de esta interfaz digital optimizada. Este método agiliza la ingesta de datos, elimina errores de formato y garantiza el cumplimiento de la *LOPD* al canalizar la información directamente hacia la base de datos *PostgreSQL* del hospital, profesionalizando un proceso que anteriormente resultaba lento y manual.

El objetivo principal era publicar esta herramienta en los servidores de *Reflex* especializados, facilitando la distribución del proyecto; no obstante, para asegurar la confidencialidad, integridad y privacidad de los datos, el proyecto tuvo que virar hacia un enfoque local. Ya que si la web funcionaba en los servidores de *Reflex Cloud*¹⁰ por óptimos que resultasen implementados los métodos de borrado, los archivos en algún momento, por mínimo que fuese se encontrarían físicamente en sus servidores; desembocando en el despliegue mediante contenedores *Docker* en la *intranet* del *HUBU*. Para garantizar la limpieza absoluta del servidor y la protección de datos, se ha implementado una arquitectura *Zero-Disk* que elimina la necesidad de vigilar carpetas locales o programar el borrado de archivos. En este modelo, la información clínica procesada reside exclusivamente de forma volátil en la memoria *RAM*, siendo purgada automáticamente por el *Garbage Collector* de *Python* al cierre de la sesión, asegurando que no quede ningún rastro físico de datos sensibles en el almacenamiento del centro.

Variables y Estudio Estadístico

Se utilizarán las variables clínicas y los indicadores de calidad establecidos (como los definidos por la *SEMICYUC*) con el objetivo de realizar la validación técnica y funcional de la herramienta web de forma adecuada.

Para cumplir los objetivos planteados en el proyecto, la evaluación de la aplicación se centrará en:

Precisión y Fiabilidad del cálculo automatizado

Validar la exactitud matemática y algorítmica de la herramienta desarrollada en *Python* para el cálculo de los indicadores de calidad. Es necesario

⁹Aquellos profesionales que desean especializarse en un área específica del cuidado de la salud y colaboran en la recolección de datos clínica.

¹⁰Servicio de despliegue en la nube del *framework Reflex*

indicar que la validación se realizará mediante la comparativa directa entre los resultados arrojados por la herramienta y el “*Gold Standard*”. En este contexto, el *Gold Standard* corresponde al cálculo manual exhaustivo de dichos indicadores realizado por el personal clínico.

Robustez y Concordancia en el procesamiento de datos

Es fundamental evaluar la capacidad de la herramienta para gestionar la limpieza de datos (valores nulos, formatos de fecha inconsistentes o errores de tipografía) sin alterar la integridad de los resultados clínicos. Para garantizar la fiabilidad de la información antes de su inserción en la base de datos *PostgreSQL*, el formulario interactivo desarrollado en *Reflex* implementa una validación exhaustiva en la entrada de datos. Este módulo controla el tipado estricto de las variables e impone la integridad estructural de los campos temporales. Asimismo, incorpora reglas lógicas que evalúan la coherencia cronológica impidiendo, por ejemplo, registrar una fecha de alta anterior a la de ingreso y establece umbrales numéricos de seguridad que restringen los valores introducidos a rangos médicamente posibles. Esta primera línea de defensa mitiga el error humano y asegura la calidad del análisis estadístico posterior.

4.3. Metodología de Ingeniería del Software

Para la conceptualización y desarrollo de esta herramienta *web*, se ha optado por implementar una **Metodología Ágil de carácter iterativo e incremental**. [?].

En el ámbito de la Ingeniería de la Salud y la informática clínica, la rigidez metodológica suele ser contraproducente. Por este motivo, se descartaron metodologías tradicionales o secuenciales, como el modelo en Cascada (*Waterfall*). Este tipo de enfoques exigen que los requisitos del sistema estén completamente definidos, documentados y cerrados desde la primera fase del proyecto. Dada la naturaleza clínica de los datos y la necesidad de validar empíricamente los cálculos de los indicadores de la *SEMICYUC*, resultaba inviable establecer unos requisitos inamovibles. El proyecto requería un marco de trabajo flexible que permitiera la adaptación rápida a los cambios, la corrección de cálculos médicos sobre la marcha y la integración continua de nuevas funcionalidades sin paralizar el desarrollo global.

Justificación Basada en el Desarrollo Real

La elección de esta estrategia ágil se fundamenta directamente en las necesidades prácticas surgidas durante el proceso de creación de la aplicación *web*:

- **Comunicación continua con el *Product Owner*:** La supervisora clínica del proyecto (la doctora responsable de la unidad de la *URCCPQ*) actuó como *Product Owner*. Gracias al enfoque iterativo, se pudieron realizar demostraciones de versiones preliminares del sistema (desde el *script* base hasta la interfaz *web* inicial), obteniendo *feedback* inmediato. Esto permitió descubrir requerimientos no contemplados inicialmente, como la inclusión de nuevas variables.
- **Desarrollo iterativo y prototipado con *Reflex*:** El sistema no se construyó de manera monolítica. Se implementó mediante iteraciones funcionales: primero se consolidó un *backend* robusto para la limpieza y procesamiento de los datos utilizando la librería *Pandas*; posteriormente, se integró el *frontend* utilizando el *framework Reflex*; a continuación, se realizó la conexión con la base de datos; y, por último, se añadieron los módulos de visualización gráfica e interactividad.
- **Gestión de incidencias y trazabilidad mediante *GitHub*:** El flujo de trabajo se gestionó a través de repositorios, documentando errores (*bugs*), refactorizaciones (como la optimización de bucles por operaciones vectorizadas para evitar la saturación de memoria) y propuestas futuras de mejora mediante el sistema de *Issues*, garantizando así el control de versiones y la planificación del *Product Backlog*.

Fases del Ciclo de Vida del Software

El ciclo de vida de la aplicación se estructuró en cinco fases iterativas principales, mostrando cuatro de ellas en la Figura 4.1 y descritas a continuación:

1. **Toma de Requisitos e Investigación:** Análisis exhaustivo de las fórmulas de calidad asistencial de la *SEMICYUC*, evaluación de viabilidad de tecnologías (descarte inicial de herramientas como *PySpark* en favor de *Python* nativo para optimizar el rendimiento local) y estudio de la estructura de los datos proporcionados por el hospital.

2. **Desarrollo del *Backend* y Lógica de Datos:** Programación de los algoritmos de carga, sistemas de validación estructurada (*First In, First Out - FIFO*), limpieza de datos atípicos, homogeneización de formatos mediante expresiones regulares, carga y adición de los datos en *PostgreSQL* y programación del núcleo matemático.
3. **Desarrollo del *Frontend* e Interfaz de Usuario (*UI*):** Creación de la arquitectura web utilizando *Reflex*. Implementación de una interfaz gráfica intuitiva alineada con los estándares visuales de la institución (*Sacyl*), diseño de ventanas dinámicas y menús de navegación.
4. **Integración y Visualización Analítica:** Conexión de la lógica con la base de datos y el componente visual. Generación algorítmica de gráficos interactivos (sectores, barras, áreas) y desarrollo de módulos de exportación estructurada para la auditoría manual de los resultados.
5. **Validación Clínica y Despliegue:** Pruebas de campo con un corte transversal real de datos (enero de 2026), implementación de protocolos de seguridad basados en arquitectura *Zero-Disk* (purga automática de datos confidenciales en *RAM* mediante *Garbage Collection*) y despliegue seguro en la *intranet* del hospital utilizando contenedores *Docker*.

Programación funcional y arquitectura modular

La arquitectura del *software* desarrollado para este proyecto se fundamenta en el paradigma de la programación funcional. Bajo este enfoque, la resolución de las distintas tareas se articula mediante la composición y el encadenamiento de funciones. Esta metodología no solo maximiza la reutilización del código, sino que resulta clave para garantizar la inmutabilidad de los datos de entrada. En su gran mayoría, el sistema se apoya en funciones puras; es decir, rutinas cuyo valor de retorno depende exclusivamente de los parámetros suministrados, evitando generar efectos secundarios o alteraciones no deseadas en las estructuras de datos originales.

Esta filosofía funcional se ha integrado estrechamente con un diseño modular, permitiendo abordar la complejidad del sistema mediante una estrategia descendente o enfoque *top-down*¹¹. De este modo, el desafío

¹¹La estrategia *top-down* es un paradigma de resolución que consiste en la segmentación sistemática de un problema complejo en fracciones menores y más manejables. En el ámbito de la ingeniería de *software*, esta técnica se traduce en la descomposición de un programa estructurado en módulos jerárquicos de alto nivel.

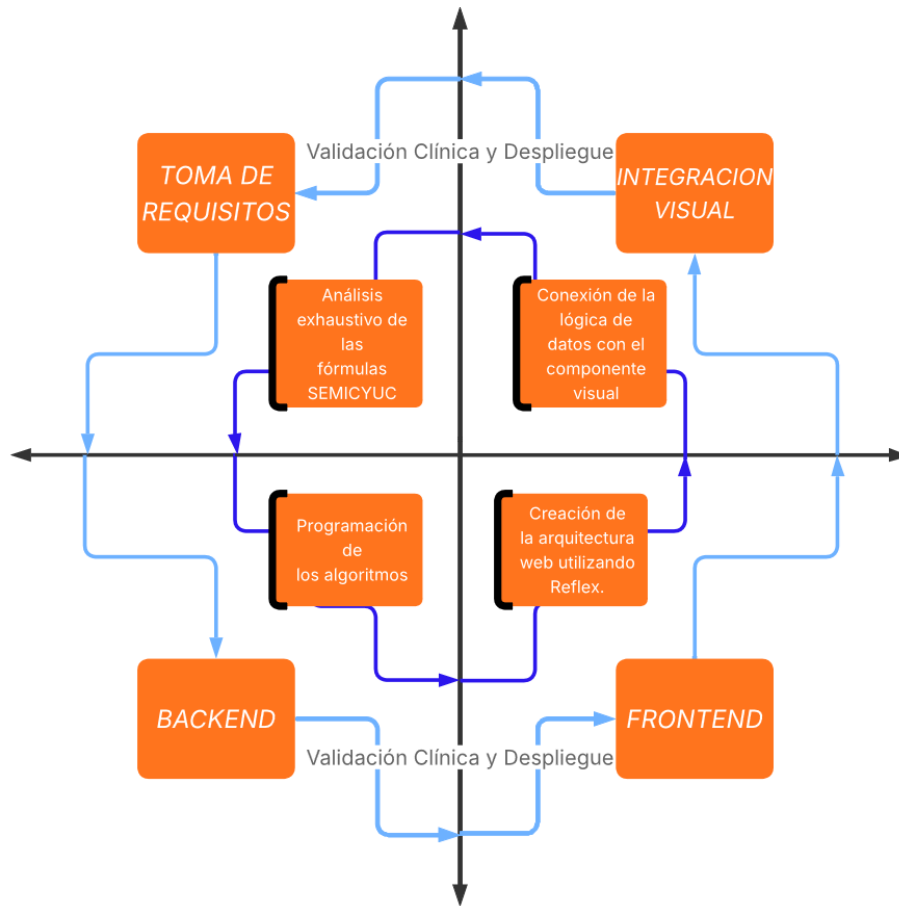


Figura 4.1: Desarrollo interactivo. *Elaboración propia*

principal se ha fragmentado en subproblemas más abarcables que se resuelven a través de módulos autónomos. Es la combinación y sinergia de los resultados de cada una de estas unidades lógicas lo que permite alcanzar el objetivo global de la aplicación.

La adopción de esta modularidad trasciende la simple reutilización algorítmica, aportando un grado de mantenibilidad crítico para el ciclo de vida del programa. Al compartimentar el sistema en unidades independientes, se simplifica drásticamente su lectura, actualización y soporte técnico. Cualquier modificación requerida en una funcionalidad concreta queda acotada a su módulo correspondiente, asegurando que el resto de la infraestructura no se vea afectada. Asimismo, esta separación estructural resulta vital durante las fases de prueba y depuración (*testing* y *debugging*), ya que permite aislar

cada componente para auditarlo individualmente, agilizando enormemente la detección y corrección de errores.

4.4. Técnicas y herramientas.

El problema planteado por la dirección de la unidad era el clásico problema de análisis de datos, en el grado de Ingeniería de la Salud se han realizado y buscado soluciones en infinidad de asignaturas. Una de ellas *Big Data*, por lo que la primera opción fue utilizar *Spark SQL*¹². Dicha asignatura mostró la utilidad y la facilidad de implementarlo en el proyecto.

En *Big Data* no se instaló *Spark SQL*, se gestionaba a través de los *Google notebooks* los cuales, corren *Ubuntu*¹³ en su *backend*, por lo que al instalar *Spark SQL* en *Windows* desembocó en infinidad de problemas:

- Problemas de controladores *USB*
- Necesidad de la instalación de *Java*
- Incompatibilidad con *Windows* de 64 *bits*
- Comandos que se habían utilizado en el *notebook* ya no funcionaban

Después de todos estos problemas se llegó a la conclusión de abandonar *Spark SQL*. Como el objetivo era usar *PostgreSQL* para guardar los datos de la unidad y después tener la opción de cargarlo en la herramienta final, se optó por usar *pandas*.

Después de solucionar el dilema de *Spark SQL* se necesitaba crear alguna herramienta o aplicación que permita a los facultativos subir sus datos y mostrar los resultados de manera sencilla. El proyecto disponía de: *Reflex* o *Streamlit*, otro *framework* de código abierto para *Python*, diseñado para crear aplicaciones *web* interactivas y visualizaciones de datos de forma rápida y sencilla. Como se ha ido relatando, la decisión final fue utilizar *Reflex*.

1. **Rendimiento y gestión del estado:** *Streamlit* recarga el código completo tras cada interacción del usuario. Por el contrario, *Reflex* emplea *WebSockets* para actualizar de forma instantánea y asíncrona únicamente las variables modificadas, optimizando la fluidez del sistema al manejar grandes volúmenes de datos históricos.

¹²Módulo para el procesamiento de datos estructurados que permite consultar información utilizando SQL

¹³Sistema operativo (SO) libre, gratuito y de código abierto basado en Linux

Entornos de programación y programas

- **Visual Studio Code (1.113.0):** Avanzado editor de código fuente disponible para *Windows*, *macOS* y *Linux*. Destaca por ser ligero y potente, ofreciendo soporte para *JavaScript*, *C++* o *Python*. [?]
- **PostgreSQL (15):** Potente sistema de gestión de bases de datos relacionales de código abierto. Destaca por su alta robustez, seguridad avanzada y cumplimiento estricto del estándar *SQL*, garantizando la persistencia e integridad de datos críticos en entornos institucionales. [?]
- **Docker Desktop (4.71.0):** Aplicación con interfaz gráfica que empaqueta todas las herramientas necesarias para construir, gestionar y ejecutar contenedores de forma rápida e intuitiva en cualquier ordenador local. [?]
- **Gemini:** Herramienta de inteligencia artificial utilizada de apoyo en el desarrollo de código implementado para el funcionamiento del proyecto. [?]
- **Lucidchart:** Herramienta en línea para la creación de diagramas y gráficos. [?]
- **GitHub:** Repositorio en línea de código fuente que facilita el control de versiones y la colaboración en proyectos software mediante el sistema *Git*¹⁴. [?]
- **GitHub Desktop (3.5.6):** Aplicación de escritorio que facilita el uso de *Git* y el trabajo con código almacenado en *GitHub*, mediante una interfaz gráfica de usuario. [?]
- **TexMaker:** Es un editor de *LaTeX*, gratuito y colaborativo, que permite redactar, editar y publicar documentos técnicos o científicos sin necesidad comprar licencias de uso. [?]
- **Microsoft Excel:** Es una aplicación de hoja de cálculo desarrollada por *Microsoft* que permite organizar, analizar, calcular y visualizar datos numéricos y de texto en filas y columnas. Es parte de la suite *Microsoft Office* y es fundamental para gestionar presupuestos, crear gráficos, realizar tareas contables y automatizar procesos con macros. [?]

¹⁴Sistema de control de versiones distribuido, que registra los cambios en el código fuente de un proyecto a lo largo del tiempo.

Lenguajes de programación

En esta sección se describirán los lenguajes utilizados para la obtención de los resultados:

- ***Python (3.12.9)***: Lenguaje de programación interpretado, de alto nivel, de propósito general, con una sintaxis legible y de tipado dinámico¹⁵. Se ha seleccionado *Python* como lenguaje de programación, no solo por las características descritas, sino también porque cuenta con una amplia biblioteca que incluye módulos para una variedad de tareas. Para su instalación [?] y es necesario añadir la extensión *Python* de *Microsoft* en *VSCode*.
- ***SQL (Structured Query Language)***: Lenguaje de dominio específico estandarizado diseñado para la gestión, consulta y manipulación de bases de datos relacionales. Proporciona una sintaxis universal y declarativa para interactuar con la información persistente, permitiendo definir estructuras, controlar accesos y recuperar registros clínicos de forma eficiente. [?]
- ***YAML y Dockerfile (Docker)***: Lenguajes de serialización de datos y configuración declarativa¹⁶ utilizados para la orquestación y contenedorización del sistema. La sintaxis nativa de *Dockerfile* se emplea para definir las instrucciones de construcción y empaquetado de la aplicación, mientras que *YAML* (*YAML Ain't Markup Language*) se utiliza en el archivo `docker-compose.yml` para configurar, interconectar y levantar simultáneamente los múltiples servicios del proyecto (el servidor *web* y la base de datos). [?]
- ***Shell Script (Bash 5.0)***: Lenguaje de *scripting*¹⁷ nativo de sistemas basados en *Unix* (*Linux*, *macOS* y *Raspberry Pi OS*). Es el lenguaje utilizado para la creación del archivo `.sh` necesario para automatizar el despliegue de la aplicación y el arranque del sistema en la “*tablet médica*”. [?]

¹⁵No es necesario especificar los tipos de datos de las variables en el momento de la declaración

¹⁶Paradigma de programación en el que se describe el estado o resultado final que se desea obtener, en lugar de detallar paso a paso cómo lograrlo.

¹⁷Secuencia de comandos

Lenguajes de programación secundarios

Aunque el desarrollo íntegro de la lógica y la interfaz gráfica de la aplicación se ha realizado en *Python*, resulta imperativo definir las tecnologías web estándar que componen el producto final desplegado. Los navegadores web carecen de la capacidad nativa para interpretar código *Python*; por ello, el *framework Reflex* actúa como un motor de compilación (*transpilador*), convirtiendo automáticamente el código fuente a la tríada fundamental del desarrollo web:

- **HTML5 (*HyperText Markup Language*)**: Lenguaje de marcado que define el esqueleto y la estructura semántica de la interfaz clínica (tales como la disposición de las tablas de pacientes, botones y contenedores). [?]
- **CSS3 (*Cascading Style Sheets*)**: Lenguaje de hojas de estilo responsable de la capa visual y del diseño adaptable (*responsive design*). Garantiza que la herramienta médica se visualice correctamente tanto en los monitores estándar de la unidad como en la pantalla táctil del terminal *Raspberry Pi*. [?]
- **JavaScript**: Lenguaje de programación del lado del cliente que dota a la interfaz de interactividad y gestiona la apertura de los *WebSockets* para comunicarse en tiempo real con el servidor *FastAPI*. [?]

Frameworks y Librerías

El desarrollo de la aplicación se ha apoyado en diversas herramientas de terceros instaladas mediante el gestor de paquetes *pip*, las cuales se dividen conceptualmente según su peso en la arquitectura del sistema:

- **Reflex (0.8.28)**: Constituye el *framework full-stack* central del proyecto. A diferencia de una librería convencional, *Reflex* define la arquitectura completa de la aplicación, orquestando de manera transparente la creación del *frontend* (*React/Vite*) y del servidor *backend* (*FastAPI*). Toda la lógica de negocio y el diseño de la interfaz médica se han estructurado bajo los paradigmas de este *framework*. [?]
- **Git**: Se ha empleado *Git* como sistema de control de versiones distribuido para mantener un registro histórico de los cambios en el código fuente. El repositorio central se ha alojado en la plataforma *GitHub* ya mencionada, garantizando la seguridad del código y facilitando su accesibilidad. [?]

- **LaTeX y BibTeX:** Lenguaje de composición de textos empleado para la redacción académica de la presente memoria, garantizando un formato tipográfico de alta calidad técnica. [?]

Para garantizar la reproducibilidad del entorno de desarrollo y documentar con precisión la arquitectura subyacente del sistema, la Tabla 4.3 detalla las versiones exactas de las librerías principales de *JavaScript* y *CSS* empleadas por el motor de compilación *Vite* [?] durante el despliegue de la aplicación.

Tecnología / Librería	Versión	Función en la Arquitectura
<i>React & React DOM</i>	19.2.3	Renderizado del árbol de componentes de la interfaz de usuario (<i>UI</i>).
<i>React Router</i>	7.13.0	Gestión del enrutamiento interno y estructura de navegación (<i>SPA</i>).
<i>Socket.IO Client</i>	4.8.3	Implementación del protocolo <i>WebSockets</i> para la comunicación asíncrona bidireccional con <i>FastAPI</i> .
<i>Recharts</i>	3.7.0	Generación y renderizado dinámico de las gráficas estadísticas y líneas de tendencia.
<i>Tailwind CSS</i>	4.1.18	Motor de generación de estilos en cascada (<i>CSS</i>) para el diseño adaptable (<i>responsive</i>).
<i>Radix UI Themes</i>	3.3.0	Base de componentes accesibles para la interacción del usuario clínico.

Tabla 4.3: Desglose de versiones del ecosistema *Frontend* autogenerado por *Reflex*.

De forma análoga al entorno cliente, el núcleo de procesamiento de datos (*backend*) requiere una declaración exhaustiva de sus dependencias. Cabe destacar que gran parte de las operaciones se han resuelto utilizando la biblioteca estándar nativa de *Python*, minimizando así la dependencia de paquetes de terceros y optimizando el rendimiento del sistema. En la Tabla 4.4 se detallan las versiones exactas del *stack* de *Python* empleado para el cálculo de los indicadores clínicos y estadísticos.

Herramientas *Hardware*

El dispositivo físico portátil, diseñado para otorgar movilidad al personal médico, opera bajo un ecosistema *software* distinto al de los equipos ofimáticos convencionales de la unidad. Como sistema operativo principal

Paquete / Librería	Versión	Función en la Arquitectura
<i>Paquetes base</i>	3.12.9	Librerías nativas (<i>asyncio</i> , <i>re</i> , <i>io</i> , <i>tempfile</i> , <i>time</i> , <i>glob</i> , <i>unicodedata</i>). [?]
<i>Pandas</i>	3.0.0	Ingesta de archivos <i>CSV</i> , limpieza de datos y estructuración en <i>DataFrames</i> para el cálculo de los indicadores <i>SEMICYUC</i> . [?]
<i>NumPy</i>	2.4.2	Soporte para operaciones matriciales y vectorización matemática [?]
<i>Scikit-Learn</i> (<i>sklearn</i>)	1.8.0	Motor estadístico y de <i>Machine Learning</i> empleado para las regresiones lineales y la obtención del coeficiente R^2 . [?]
<i>Matplotlib</i>	3.10.8	Motor de renderizado gráfico secundario para operaciones en el servidor. [?]
<i>FPDF2</i>	2.8.7	Generación programática y exportación de informes clínicos en formato <i>PDF</i> . [?]
<i>Psycopg2-Binary</i>	2.9.11	Adaptador a bajo nivel encargado de la comunicación física y ejecución de transacciones con el servidor <i>PostgreSQL</i> . [?]
<i>Alembic</i>	1.18.4	Se encarga de traducir las modificaciones de los modelos de datos en <i>Python</i> a código <i>DDL</i> y delega en <i>Psycopg2</i> su ejecución física para alterar la estructura de las tablas en <i>PostgreSQL</i> . [?]
<i>Bcrypt</i>	5.0.0	Implementación de funciones de derivación de claves criptográficas (<i>hashing</i>) para el almacenamiento seguro de contraseñas. [?]

Tabla 4.4: Desglose de versiones del ecosistema *Backend* (Entorno *Python*).

se ha implementado *Raspberry Pi OS (64-bit)* [?], una distribución oficial basada en el núcleo *Linux* (arquitectura *Debian*).

La elección de este sistema operativo se fundamenta en su alta estabilidad, su optimización nativa para procesadores *ARM* y su eficiencia en la gestión de recursos limitados. A nivel de ingeniería de despliegue, la naturaleza multiplataforma de *Python* y *Reflex* ha permitido que el mismo código fuente se ejecute sin alteraciones en este entorno (solamente la aplicación en *Reflex*, sin una conexión a *PostgreSQL*).

La automatización del arranque en la *tablet* médica se gestiona mediante un *script* nativo de *Bash* (*.sh*) [?]. Esto permite que, al encender el dispositivo,

el servidor *FastAPI* y la interfaz gráfica se inicialicen de forma autónoma, ofreciendo al facultativo un acceso directo a la herramienta clínica sin requerir interacción previa con el sistema operativo subyacente.

Resultados

Una vez finalizado el desarrollo y explicación del proyecto, es necesario realizar un análisis de los resultados. Comprobando la eficiencia y eficacia del trabajo realizado en comparación con los objetivos planteados al inicio.

5.1. Resumen de resultados

El trabajo realizado en este proyecto desembocó en una aplicación *web* profesional y resolutive. Dicha *web* está construida en su totalidad en *Reflex* y cuenta con las funciones necesarias para el cálculo de indicadores deseados por la jefatura de la unidad.

Esta plataforma cuenta con un módulo de carga de datos que admite la subida de un máximo de diez archivos *CSV* o, alternativamente, la extracción directa de un histórico clínico de hasta diez años desde la base de datos, permitiendo así el seguimiento de un amplio espectro temporal, en la siguiente imagen 5.1 se pueden observar los módulos.

El gestor de archivos interno se basa en una *FIFO*¹⁸ generando un mensaje por pantalla de todos los archivos/datos eliminados en el proceso de carga.

Una vez se han cargado los registros es posible seleccionar un botón de borrado, que como su nombre indica se encarga de eliminar los datos alojados en la memoria *RAM*. Cuenta con un menú de selección de indicadores que facilita el cálculo de los veintiún indicadores, mostrando un gráfico lineal o de barras a decisión del personal visible en la siguiente imagen 5.2, que

¹⁸Estrategia de gestión de almacenamiento donde los elementos más antiguos o introducidos primero son los primeros en eliminarse

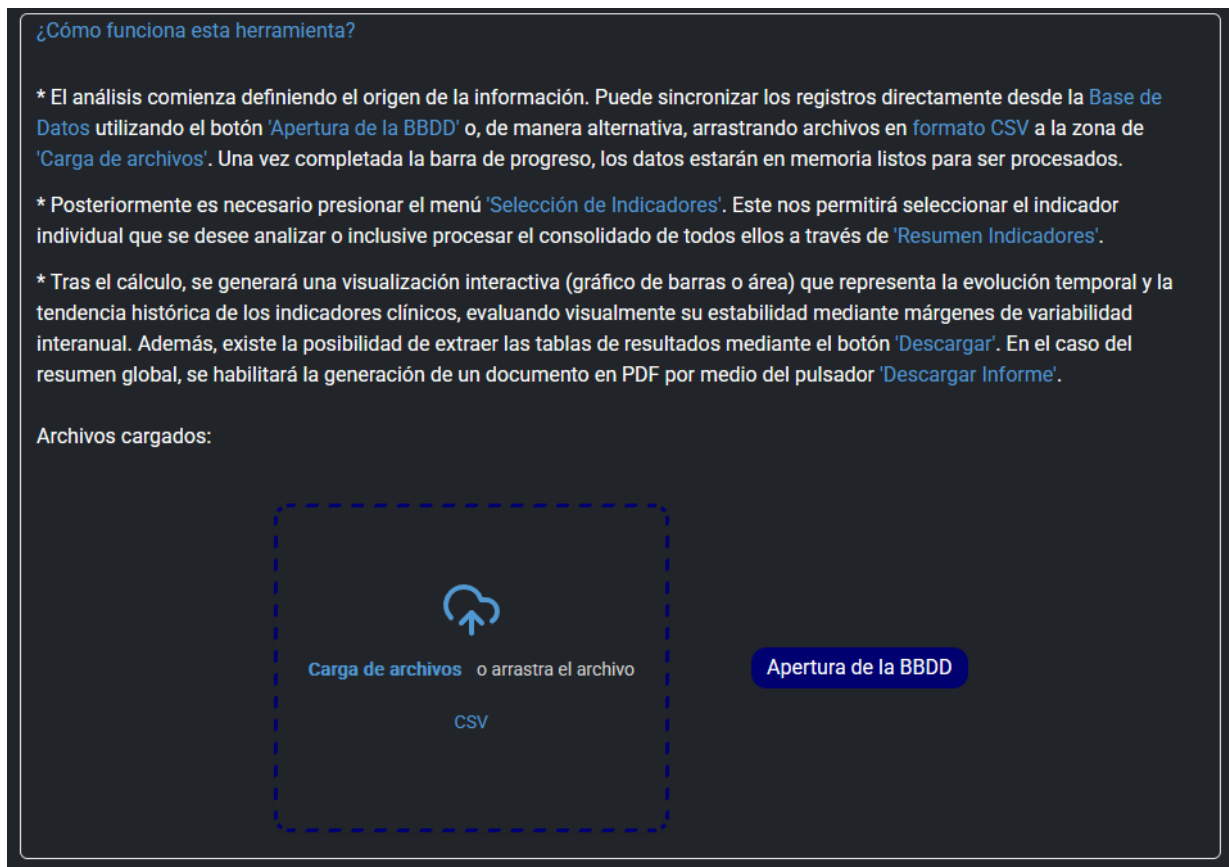


Figura 5.1: Módulos principales de carga/extracción de datos.

representan la evolución temporal y la tendencia histórica de los indicadores clínicos de la URCCPQ, evaluando visualmente su estabilidad mediante márgenes de variabilidad interanual. Por último cuenta con cuatro utilidades adicionales, dos relacionadas con la base de datos y otras dos con el análisis de los registros:

1. **Gestión de registros clínicos:** Permite la administración integral de la información en la base de datos. Para garantizar la consistencia de los datos y prevenir duplicidades, el sistema requiere realizar obligatoriamente una búsqueda previa del paciente antes de habilitar las funciones para añadir, editar o borrar sus registros. En la siguiente imagen [5.3](#) se puede observar el panel de gestión de la *BBDD*.
2. **Exportación de histórico clínico:** Habilita la extracción de los registros correspondientes a múltiples años en formato *CSV*. La des-

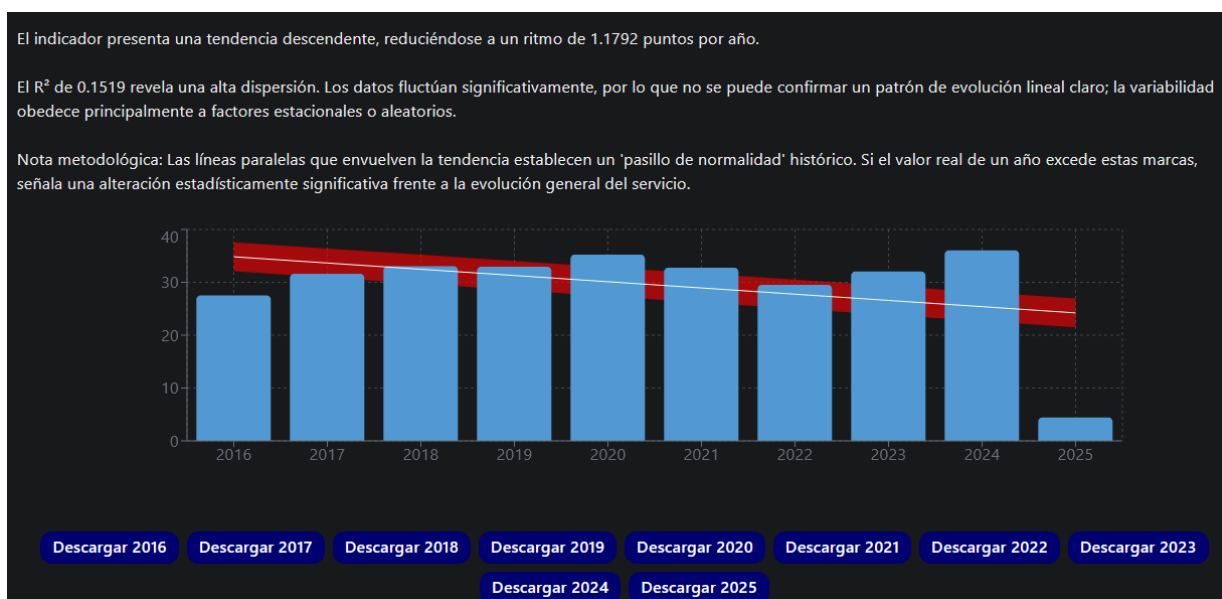


Figura 5.2: Gráfico de barras calculado por la aplicación.

Base de Datos Clínicos

Explorador de registros completos. Deslice lateralmente para ver todas las columnas.

Seleccione los años a procesar (Máx 10):

2026 2025 2024 2023 2022 2021 2020 2019 2018 2017 2016

Buscar por N° Historia... Ej. 111111 Buscar Registro + Añadir Registro

Borrado	Edición	N° Historia	Fecha Ingreso	Fecha Alta	Reingreso 48h	Especialidad	Ingreso Urgencia	Fallecimiento	APACHE	VMi	Días VM
		H-2016-2	16/03/2016	29/03/2016	False	Cardiología	—	False	12.0	—	7.4

Cargar para Análisis Cancelar

Figura 5.3: Panel de la BBDD.

carga se genera de forma totalmente anonimizada, garantizando el cumplimiento de la *LOPD* y permitiendo al personal médico realizar el análisis estadístico en el terminal portátil (*Raspberry Pi*) de forma segura fuera del entorno hospitalario.

3. **Mezcla de indicadores:** Permite combinar un máximo de tres indicadores con el fin de permitir su visualización en un mismo gráfico.

Esta función facilitara la comparación de aquellos que se encuentren relacionados, en la imagen 5.4 se muestra un ejemplo.

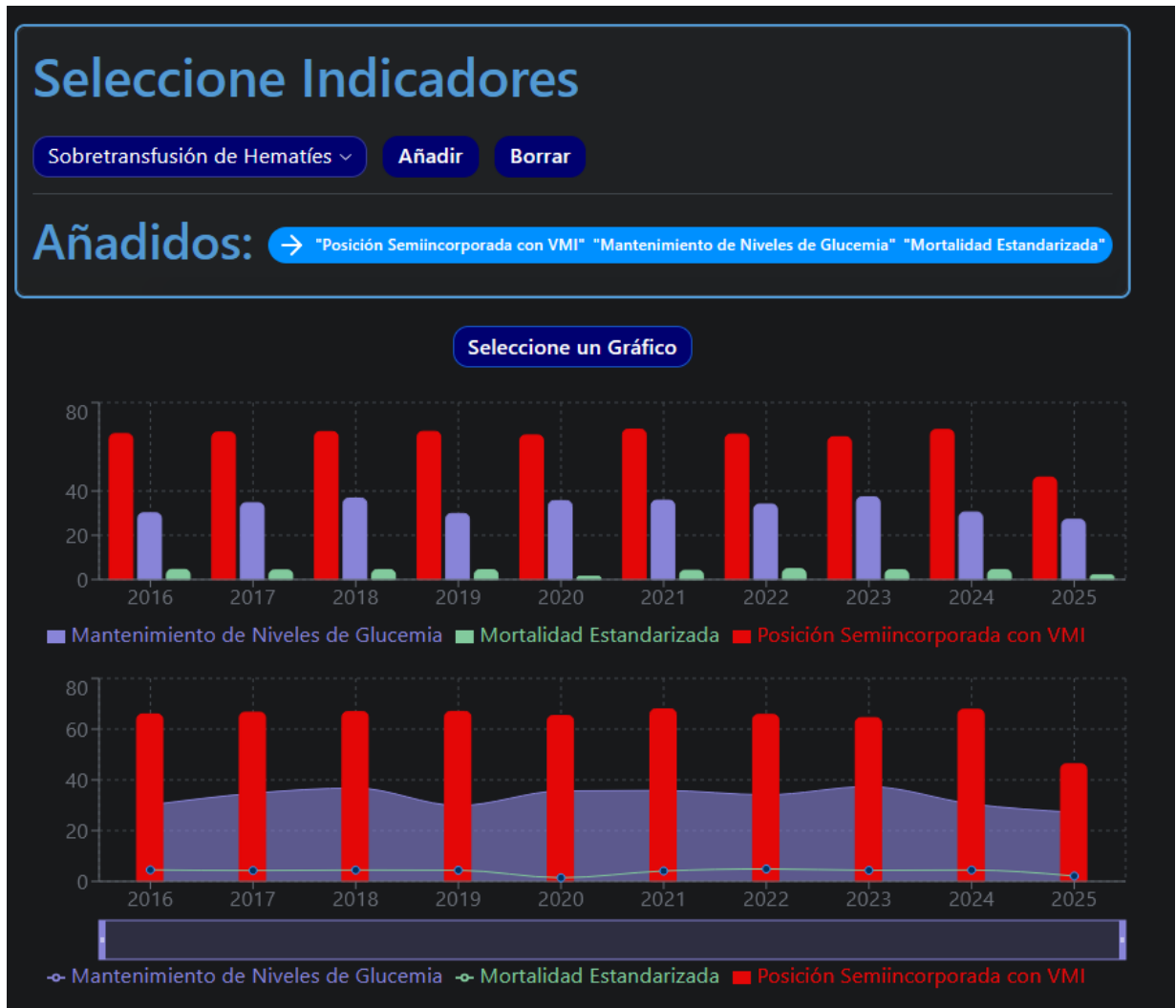


Figura 5.4: Combinación de 3 indicadores.

- Resumen Indicadores:** Muestra dos tablas, la primera representa el resumen de los indicadores, mientras que la segunda hace referencia a los porcentajes de ingresos por especialidades, ambas descargables. Cuenta con una tercera funcionalidad, la cual permite descargar un informe final con todos los indicadores y sus respectivos gráficos, en la siguiente imagen 5.5 se muestran los aspectos descritos en este apartado.



Figura 5.5: Resumen de los indicadores.

Todos los procesos descritos anteriormente son realizables por cualquier usuario gracias al cumplimiento de todos los objetivos que se plantearon. 5

- Se realizó un diseño agradable, intuitivo y simple de utilizar para los facultativos, permitiendo que con unos pocos *clicks* se puedan realizar todos los procesos que ofrece la plataforma.
- El despliegue de la aplicación en la unidad se realiza de forma segura mediante contenedores *Docker* en la *intranet* del hospital. Para facilitar el acceso desde el terminal portátil (*Raspberry Pi*), se ha desarrollado un *script* ejecutable (*.sh*) que automatiza todo el proceso de arranque de la herramienta.
- La recolección de los datos inició en el momento que el proyecto recibió el visto bueno por parte del comité de bioética del *HUBU*. Permitiendo

a la jefatura de la unidad disponer de ellos en cualquier comento a partir de la fecha de aprobación.

A pesar de que no era un objetivo principal, se ha logrado un desarrollo sumamente satisfactorio en el prototipo *hardware*. El dispositivo obtenido facilita la movilidad del usuario al momento del cálculo permitiendo a la dirección de la unidad mostrar los resultados de su actuación clínica sin necesidad de encontrarse físicamente en la *URCCPQ*, presentando una autonomía completa, asegurando la seguridad y manejabilidad del prototipo.

Discusión

Los resultados obtenidos en este Trabajo de Fin de Grado representan una innovación significativa respecto a los métodos tradicionales de gestión clínica en la unidad de Reanimación (*URCCPQ*). Uno de los mayores avances es la automatización integral del procesamiento de datos mediante una interfaz *web* especializada, diseñada específicamente para aislar al personal sanitario de la complejidad técnica subyacente. A diferencia de los registros estáticos o las hojas de cálculo manuales, la plataforma transforma grandes volúmenes de datos brutos en visualizaciones gráficas y estadísticas comprensibles e interactivas. Esta característica diferenciadora sitúa a la herramienta no solo como un repositorio digital, sino como un sistema de apoyo activo para la toma de decisiones informadas, permitiendo a la dirección médica evaluar la evolución clínica basándose en los estándares oficiales de la *SEMICYUC*.

Dada la criticidad del entorno hospitalario al que va dirigido, la evaluación del éxito de este proyecto trasciende las métricas convencionales de usabilidad de *software*. Para confirmar la solidez del sistema y el cumplimiento íntegro de los objetivos planteados, es necesario analizar el producto desde una perspectiva multidimensional que abarque tanto su integración física en la unidad como la exactitud de sus procesos internos. Por consiguiente, a continuación se detallan los resultados del proyecto estructurados en dos ejes fundamentales: el rigor analítico de su motor algorítmico y su valor traslacional a la práctica médica diaria.

6.1. Validación y Explicación Matemática

Justificación Metodológica del Cálculo de Variabilidad y Error Estadístico

- **Agregación de datos y tamaño muestral:** En el presente proyecto, el cálculo de los indicadores se realiza mediante la agregación anual de los datos brutos extraídos del sistema de información hospitalaria. Esto significa que la totalidad de los episodios clínicos de un año natural se consolida en un único valor final. En consecuencia, para cada año y especialidad evaluada, el tamaño de la muestra temporal es de un único punto de datos ($N = 1$).
- **Imposibilidad matemática del cálculo de dispersión intra-anual:** Debido a esta arquitectura de datos, resulta matemáticamente imposible calcular una medida de dispersión estadística interna (como la varianza o la desviación estándar) para un año aislado. La fórmula estadística de la varianza muestral requiere dividir la suma de las diferencias al cuadrado entre los grados de libertad ($N - 1$):

$$S^2 = \frac{\sum_{i=1}^N (x_i - \bar{x})^2}{N - 1} \quad (6.1)$$

Al trabajar con indicadores consolidados anualmente ($N = 1$), el denominador de la ecuación resulta en cero ($1 - 1 = 0$), generando una indeterminación matemática. Para calcular un error específico por año, sería metodológicamente necesario fraccionar la variable temporal (por ejemplo, en cortes mensuales). Sin embargo, esto desviaría el objetivo de la herramienta, la cual está diseñada para evaluar tendencias de gestión hospitalaria a nivel macro (anual) y no para analizar la estacionalidad o el “ruido” estadístico a corto plazo.

- **Aplicación del Error Típico de la Estimación Histórica:** Para solventar esta limitación y dotar a la visualización gráfica de rigor analítico, la métrica de variabilidad (representada visualmente como una franja sombreada o “pasillo de confianza” continuo) implementada en el panel no refleja la desviación estándar interna de un año en concreto, sino el Error Típico de la Estimación derivado del modelo de tendencia general de la serie histórica completa.

Este enfoque metodológico permite hacer una estimación visual. Si el valor real de un indicador en un año determinado excede los límites de esta franja, el sistema evidencia que dicha fluctuación es

estadísticamente significativa y escapa al comportamiento histórico normal del servicio. Esta aproximación resulta de alto valor clínico y de gestión, ya que permite identificar rápidamente anomalías reales frente a fluctuaciones habituales.

Tratamiento de Microfluctuaciones y Criterio de Estabilidad Tendencial

- **Diferenciación entre Ruido Estadístico y Tendencia Real:** En el análisis de series temporales de indicadores de gestión hospitalaria, es habitual encontrar variables que presentan fluctuaciones interanuales mínimas. Desde un punto de vista puramente matemático, cualquier regresión lineal forzará el cálculo de una pendiente ($m \neq 0$) adaptándose a estas variaciones, por mínimas que sean. Sin embargo, desde una perspectiva clínica y de gestión, variaciones centesimales entre años representan mero “ruido estadístico” inherente al funcionamiento del hospital, y no un cambio estructural en el servicio.
- **Implementación del Criterio de Amplitud Mínima:** Para evitar la generación de “falsas tendencias” visuales que puedan inducir a error en la toma de decisiones, el algoritmo de este proyecto implementa un filtro de significancia basado en la amplitud del rango histórico. Se calcula la diferencia neta entre el valor máximo y el valor mínimo registrado en la serie de tiempo para cada indicador:

$$\Delta = \text{Valor}_{\max} - \text{Valor}_{\min} \quad (6.2)$$

Si esta amplitud (Δ) no supera un umbral clínico predefinido (es decir, la variación es estadísticamente y clínicamente irrelevante), el sistema descarta el modelo de regresión inclinada.

- **Aplicación del Modelo de Pendiente Nula (Sistema Estable):** Cuando se detecta que el indicador carece de variabilidad significativa, el algoritmo proyecta una línea de tendencia totalmente horizontal, matemáticamente equivalente a la media histórica del periodo ($y = \bar{x}$). Esta decisión metodológica aporta un alto valor al cuadro de mando, comunicando visualmente al usuario que el servicio se encuentra en un estado de “estabilidad y control”, evitando que los gestores sobre-reaccionen ante pendientes visuales que, en la realidad del volumen hospitalario, representan fluctuaciones de impacto nulo.

6.2. Impacto de la Herramienta

El cálculo, monitorización y análisis de estos indicadores resultan de vital importancia para el correcto funcionamiento de la Unidad y el beneficio directo de los pacientes, fundamentándose en los siguientes puntos clave:

- **Impacto directo en la Seguridad del Paciente (Principio de No Maleficencia):** La extracción y evaluación de estos indicadores permite la detección temprana de eventos adversos, complicaciones o desviaciones en los protocolos clínicos. Al disponer de una herramienta que automatiza este análisis, la Unidad puede identificar áreas de mejora de forma ágil, reduciendo drásticamente los riesgos asociados a la práctica clínica y previniendo futuros incidentes.
- **Mejora Continua de la Calidad Asistencial (Principio de Beneficencia):** El uso de métricas objetivas (alineadas con los estándares de sociedades científicas como la *SEMICYUC*) es el único método validado para auditar la calidad del servicio. Transformar los datos crudos almacenados en el sistema *ICCA* en información estructurada dota a los facultativos de una retroalimentación real sobre la eficacia de los tratamientos aplicados, permitiendo optimizar las guías de práctica clínica.
- **Optimización de Recursos Críticos (Justicia Distributiva):** Las unidades de críticos y de alta complejidad son entornos donde los recursos (camas, aparataje, personal especializado) son finitos y de altísimo coste. Analizar estos indicadores permite comprender los flujos de pacientes, predecir tiempos de estancia y optimizar las altas, garantizando que los recursos del hospital estén siempre disponibles para los pacientes que más lo necesitan.
- **Toma de Decisiones Basada en la Evidencia:** Históricamente, el análisis retrospectivo de datos en la unidad ha supuesto una carga burocrática inasumible, limitando la capacidad de investigación. La sistematización de estos indicadores mediante la presente herramienta de software permite a los clínicos tomar decisiones estratégicas basadas en la evidencia empírica de su propia unidad, y no únicamente en la intuición o en estudios externos.

Aspectos clave de la Herramienta:

1. **Hiper-especialización:** Esta *app* no es un *Excel* genérico, es un software que tiene embebida la lógica médica clínica española (*SEMICYUC*). Sabe exactamente qué es un reingreso no programada o cómo normalizar las especialidades de un hospital.
2. **Automatización del trabajo sucio:** Cubre el hueco exacto que los grandes programas ignoran: el paso intermedio. El *ICCA* facilita datos incomprensible que mi aplicación se encarga de transformar en gráficos y tablas de auditoria con un solo clic.
3. **Independencia y bajo coste:** Es una solución de código abierto construida con *Reflex* y *Pandas*. No requiere de licencias de miles de euros y se despliega eficientemente mediante contenedores *Docker* en la *intranet* del *HUBU*. Esta arquitectura asegura que los datos sensibles nunca salgan a internet, garantizando el aislamiento total de la información clínica y el cumplimiento estricto de la normativa de seguridad del centro.
4. **Trazabilidad y Control de Accesos:** El sistema integra un motor de persistencia mediante una base de datos relacional (*PostgreSQL*) que gestiona de forma segura la autenticación de los facultativos. Además, incorpora un registro de auditoría inalterable que documenta cada acción, modificación o extracción de datos realizada, garantizando la trazabilidad absoluta de las operaciones y protegiendo legalmente al centro ante el estricto cumplimiento de la *LOPD*.

En conclusión, el acceso y tratamiento de estos datos (protegidos en todo momento mediante el despliegue de una arquitectura estrictamente local) no responde a un mero ejercicio estadístico, sino a una necesidad imperativa para garantizar una atención médica más segura, eficiente, ética y de la más alta calidad para los futuros pacientes.

6.3. Desafíos Técnicos y Evolución Arquitectónica

Durante el ciclo de vida del desarrollo de la plataforma para la unidad (*URCCPQ*), se presentaron diversos obstáculos técnicos e imprevistos metodológicos que requirieron la reevaluación continua y la adaptación de las

estrategias iniciales. La necesidad de integrar una herramienta analítica en un entorno clínico restrictivo obligó a iterar sobre el diseño del sistema. A continuación, se detallan las principales problemáticas enfrentadas y las soluciones de ingeniería implementadas.

Ingeniería de Requisitos y Selección de la Pila Tecnológica

El inicio de este proyecto supuso un reto de ingeniería de requisitos considerable. Al estar dirigido a facultativos médicos, cuyo perfil carece de formación técnica específica en informática, la problemática se planteó inicialmente de forma sumamente abierta, sin una arquitectura predefinida ni limitaciones tecnológicas claras. Esta abstracción obligó a asumir el liderazgo del diseño desde su concepción, requiriendo traducir necesidades médicas complejas y protocolos críticos (como la escala *APACHE* o los días de *VMI*) en lógica computacional.

El primer gran desafío derivado de esta fase fue la selección del entorno de desarrollo. La necesidad de aunar un potente motor estadístico con una interfaz *web* moderna e interactiva generó dificultades a la hora de determinar qué lenguaje y *framework* emplear. Tras evaluar distintas alternativas, se optó por unificar el desarrollo bajo *Python* empleando *Reflex*. Esta decisión implicó enfrentarse a la curva de aprendizaje de tecnologías de despliegue novedosas, pero resultó una solución vital para superar las severas limitaciones estructurales y visuales que presentaban otras opciones más tradicionales de la ciencia de datos.

Limitaciones de Despliegue Local: Ejecutables frente a Contenedores

El segundo aspecto crítico, y uno de los mayores escollos del proyecto, tuvo lugar durante la fase de despliegue local. Con el objetivo de facilitar el uso a los doctores, el planteamiento original buscaba compilar y empaquetar toda la aplicación en un único archivo ejecutable tradicional (*.exe*). Sin embargo, la complejidad arquitectónica de un *framework full-stack* hizo descartar esta alternativa por incompatibilidades críticas fundamentadas en los siguientes factores:

- **Arquitectura distribuida y WebSockets:** *Reflex* opera bajo un modelo *full-stack* que despliega dos servidores simultáneos (*Frontend*

y *Backend*). La sincronización del estado de la aplicación y la transferencia de datos entre ambos dependen de una conexión bidireccional continua a través de *WebSockets*.

- **Ejecución en directorios temporales:** El empaquetado en un único archivo obliga a que el código se descomprima dinámicamente en una carpeta temporal del sistema anfitrión cada vez que se ejecuta.
- **Restricciones de seguridad del sistema operativo:** Los *firewalls* y sistemas de protección nativos (como *Windows Defender*) aplican políticas estrictas sobre los procesos iniciados desde rutas temporales. Al intentar levantar servidores locales y abrir puertos de red desde estas ubicaciones, el sistema operativo interpreta la acción como una posible amenaza, bloqueando silenciosamente el tráfico de los *WebSockets*.

Como resultado de este bloqueo, la interfaz gráfica lograba renderizarse en el navegador, pero la comunicación con la lógica del servidor (despliegue de eventos, subida de archivos) quedaba completamente anulada. Como solución de ingeniería, se rediseñó por completo la estrategia de despliegue mediante la orquestación de contenedores *Docker* directamente en la *intranet* del hospital. Esta alternativa garantizó el correcto flujo de red y el estricto respeto a las políticas de seguridad institucionales, proporcionando a los facultativos la misma experiencia de acceso *Zero-Install* y *plug and play* que se buscaba inicialmente sin comprometer la funcionalidad bidireccional.

Evolución de la Persistencia: Transición hacia Modelos Relacionales

Finalmente, un tercer desafío técnico relevante fue la evolución del sistema de gestión de la información. El planteamiento analítico original contemplaba una persistencia volátil basada exclusivamente en la carga manual de archivos planos de tipo *CSV*.

No obstante, durante el desarrollo se identificó que este enfoque carecía de la trazabilidad necesaria para un entorno médico. Con el fin de profesionalizar la herramienta, garantizar la integridad de los registros a largo plazo y permitir un manejo de la información auditable, se planteó el reto de migrar hacia la implementación de una base de datos relacional sin afectar al motor de cálculo ya construido.

Para solventar esta carencia, se seleccionó e integró *PostgreSQL*. Esta decisión estuvo motivada estratégicamente por la necesidad de asegurar la

compatibilidad nativa con la infraestructura tecnológica ya operativa en el servidor central del *HUBU*. Este cambio arquitectónico permitió abandonar el manejo de archivos locales aislados en favor de una solución integrada (*SQLModel* acoplado a *Pandas*), capaz de soportar el histórico clínico de la unidad bajo los estándares de seguridad, concurrencia y rendimiento exigidos por el hospital.

Conclusiones

La ejecución del presente Trabajo de Fin de Grado ha culminado con el desarrollo y despliegue de un *software* innovador para la gestión, monitorización y cálculo de indicadores clínicos en la unidad de Reanimación (*URCCPQ*), cumpliendo y superando los objetivos propuestos al inicio del proyecto.

La verdadera naturaleza de este trabajo trasciende el mero desarrollo de *software*, justificando plenamente su pertinencia dentro del ámbito de la **Ingeniería de la Salud**. El diseño de esta plataforma ha exigido ir mucho más allá del código: ha requerido comprender el lenguaje médico, la criticidad de variables de soporte vital (como la escala *APACHE* o los días de *VMI*) y los exigentes protocolos clínicos. Esta capacidad para empatizar con el flujo de trabajo del facultativo y traducir necesidades médicas complejas en soluciones tecnológicas usables y seguras es el valor diferencial aportado en este trabajo.

En consonancia con los objetivos planteados, se enumeran a continuación las conclusiones relativas a su grado de cumplimiento y los hitos técnicos alcanzados:

1. **Consecución del objetivo general: Automatización y mejora del flujo clínico.** Se ha desarrollado y validado con éxito la plataforma *web* clínica. La herramienta ha transformado una necesidad abstracta y cálculos manuales en un sistema de grado de producción que automatiza el análisis del desempeño de la unidad, reduciendo drásticamente la carga administrativa de los facultativos médicos.

2. **Cumplimiento de la integración y seguridad del sistema.** Se ha logrado un despliegue local, seguro y centralizado en la *intranet* del hospital, prescindiendo de la nube (*Reflex Cloud*).
 - Para lograr este hito, se superó un escollo arquitectónico crítico: el intento inicial de empaquetado en un ejecutable (.exe) fue bloqueado por las políticas de seguridad del sistema operativo sobre los *WebSockets*. Esto se solventó exitosamente mediante la orquestación de contenedores *Docker*, garantizando el aislamiento exigido.
 - Asimismo, se estableció el flujo de recolección semiautomática mediante la integración de una base de datos relacional *PostgreSQL*, abandonando el manejo exclusivo de archivos planos iniciales y asegurando la persistencia y auditoría del histórico clínico.
3. **Cumplimiento del desarrollo *software* (Procesamiento y Accesibilidad).** El motor analítico construido bajo *Python* cumple rigurosamente con la *LOPD* gracias a la exitosa implementación de la arquitectura *Zero-Disk*. Los registros se extraen de la base de datos y se inyectan en *DataFrames* de *Pandas* utilizando *buffers* temporales en memoria *RAM*, garantizando su destrucción por el *Garbage Collector* al cerrar la sesión. A su vez, se diseñó la interfaz *Zero-Install* accesible, permitiendo también procesar de forma segura archivos *CSV* locales.
4. **Cumplimiento del desarrollo *hardware* (Terminal autónomo).** Se ha alcanzado el objetivo de versatilidad mediante la construcción y optimización del prototipo basado en la arquitectura *Raspberry Pi* (“*tablet médica*”). Esto ha dotado al personal de un entorno *plug and play* portátil, permitiendo el uso aislado del sistema y la descarga de registros anonimizados sin riesgo de vulnerar la red central del hospital.

Como conclusión final, el sistema definitivo optimiza y simplifica el procesamiento de datos en la unidad, garantizando un acabado profesional y sentando unas bases tecnológicas sólidas y escalables para futuras implementaciones hacia un ecosistema integral de monitorización hospitalaria.

Lineas de trabajo futuras

El presente proyecto ha logrado satisfacer con creces los objetivos planteados, culminando en una herramienta de grado de producción completamente funcional para la unidad (*URCCPQ*). No obstante, el diseño de la arquitectura de *software* se ha concebido bajo el principio de escalabilidad, sentando unas bases sólidas que permiten futuras iteraciones hacia un ecosistema de integración clínica absoluta.

En cuanto al flujo de información, la línea de trabajo futuro más ambiciosa y con mayor impacto clínico es la automatización total de la ingesta de datos. En su versión actual, la herramienta requiere que los facultativos introduzcan manualmente los registros clínicos en la base de datos *PostgreSQL* a través de los formularios de la interfaz. El objetivo a medio plazo es desarrollar conectores directos a nivel de base de datos o módulos de interoperabilidad (*APIs*) que integren la aplicación directamente con el sistema de información clínica *ICCA* (*IntelliSpace Critical Care and Anesthesia*) utilizado en la unidad. Esta integración permitiría alimentar el motor algorítmico de *Python* en tiempo real extrayendo la información de origen.

Finalmente, la consolidación de esta conexión automatizada con *ICCA* abriría la puerta a expandir el motor estadístico actual. Aprovechando librerías como *Scikit-Learn*, ya integradas en el proyecto, se podrían entrenar modelos predictivos de *Machine Learning*¹⁹ capaces de alertar sobre desviaciones en los indicadores antes de que ocurran, transformando la aplicación de una herramienta de monitorización y análisis a un verdadero sistema inteligente de soporte a la decisión clínica.

¹⁹Rama de la Inteligencia Artificial que permite a las máquinas aprender y mejorar de forma autónoma a través de datos y algoritmos, sin ser programadas explícitamente.

Bibliografía

- [phi, a] IntelliSpace Critical Care and Anesthesia — philips.com.sg. URL: <https://www.philips.com.sg/healthcare/product/HCNOCTN332/intellispacecriticalcareandanesthesia>.
- [phi, b] Philips - IntelliSpace Cuidados Críticos y Anestesia (ICCA) — philips.es. URL: <https://www.philips.es/healthcare/product/HCNOCTN332/intellispace-para-cuidados-crticos-y-anestesia-icca>.
- [Alembic,] Alembic. Welcome to alembic's documentation! URL: <https://alembic.sqlalchemy.org/en/latest/>.
- [Bash,] Bash. Bash. URL: <https://es.wikipedia.org/wiki/Bash>.
- [Bcrypt,] Bcrypt. A simple tool to generate and verify bcrypt hashes. all processing happens in your browser for security. URL: <https://bcrypt-generator.com/>.
- [Bolaki M*, 2023] Bolaki M*, Papakitsou I, M. V. K. E. (2023). Electronic health record in the icu: An essential need in the modern era. *Archives of Case Reports*, page 029–031. Estudio sobre las barreras de implementación, rigidez y desafíos de adaptación de los sistemas comerciales de historia clínica electrónica en las unidades de cuidados intensivos. DOI: <https://doi.org/10.29328/journal.acr.1001072>.
- [Cruz, 2020] Cruz, P. T. (2020). Coeficiente de error típico. URL: <https://www.uv.mx/iiesca/files/2013/01/coeficiente2005-1.pdf>.
- [CSS3,] CSS3. Qué es css3 y sus fundamentos. URL: <https://openwebinars.net/blog/que-es-css3/>.

- [de ISO, 2015] de ISO, S. C. (2015). Sistemas de gestión de la calidad — requisitos. URL: https://repositorio.buap.mx/rcontraloria/public/inf_public/2019/0/NOM_ISO_9001-2015.pdf.
- [de Sanidad, 2026] de Sanidad, M. (2026). Historia clínica del sistema nacional de salud (hcdsns). URL: <https://www.sanidad.gob.es/areas/saludDigital/historiaClinicaSNS/home.htm>.
- [Delgado, 2017] Delgado, M. C. M. (2017). Indicadores de calidad en el enfermo crítico. URL: https://semicyuc.org/wp-content/uploads/2018/10/indicadoresdecalidad2017_semicyuc_spa-1.pdf.
- [Desktop, a] Desktop, D. The 1 containerization software for developers and teams. URL: <https://www.docker.com/products/docker-desktop/>.
- [Desktop, b] Desktop, G. Github desktop. URL: <https://desktop.github.com/download/>.
- [Europea, 2023] Europea, U. (2023). Diferencia entre front end y back end. URL: <https://universidadeuropea.com/blog/diferencia-front-end-y-back-end/>.
- [Europea, 2025] Europea, U. (2025). Análisis de tendencias. URL: <https://universidadeuropea.com/blog/analisis-tendencias/>.
- [Excel,] Excel. Microsoft excel. URL: <https://www.microsoft.com/es-es/microsoft-365/excel>.
- [Fatimetou Zahra, 2019] Fatimetou Zahra, N. A. M. (2019). The business intelligence use in healthcare and its enhancement by predictive analytics. *International Journal of Computer Trends and Technology*, 67(7). Análisis de las limitaciones de las plataformas BI genéricas en salud, destacando la alta curva de aprendizaje y los complejos procesos ETL (Extract, Transform, Load) requeridos. URL: <https://www.ijcttjournal.org/Volume-67%20Issue-7/IJCTT-V67I7P105.pdf>.
- [Fpdf2,] Fpdf2. Simple and fast pdf generation for python. URL: <https://pypi.org/project/fpdf2/>.
- [Gemini,] Gemini. Gemini google. URL: <https://gemini.google.com/app>.
- [Git,] Git. Git—everything-is-local. URL: <https://git-scm.com/>.
- [GitHub,] GitHub. Github. URL: <https://github.com/>.

- [Hat, 2023] Hat, R. (2023). Yaml: qué es y usos. URL: <https://www.redhat.com/es/topics/automation/what-is-yaml>.
- [HTML5,] HTML5. Python software foundation. URL: <https://es.wikipedia.org/wiki/HTML5>.
- [hubspot, 2025] hubspot (2025). Qué es frontend y backend. URL: <https://blog.hubspot.es/website/frontend-y-backend>.
- [IBM, 2025] IBM (2025). ¿qué es una api rest? URL: <https://www.ibm.com/es-es/think/topics/rest-apis>.
- [IBM, 2026] IBM (2026). Websocket. URL: <https://www.ibm.com/docs/es/was/9.0.5?topic=applications-websocket>.
- [Inc., 2024] Inc., D. (2024). Docker: Accelerated container application development. URL: <https://www.docker.com/>.
- [JavaScript,] JavaScript. Que es javascript. URL: <https://lenguajejs.com/javascript/>.
- [Jiménez, 2018] Jiménez, F. J. F. (2018). Programación shell-script en linux. URL: <http://trajano.us.es/~fjffj/shell/doc/>.
- [Latex,] Latex, B. Gestión de bibliografía bibtex - una guía detallada para latex. URL: <https://bibtex.eu/es/>.
- [Lucidchart,] Lucidchart. Lucidchart. URL: <https://www.lucidchart.com/pages/es>.
- [Matplotlib,] Matplotlib. Matplotlib - visualization with python. URL: <https://matplotlib.org/>.
- [Muñoz et al., 2024] Muñoz, M. F., de la Torre, J. J., López, S. P., Sierra, S. H., and Uribe, C. A. C. (2024). Estudio comparativo de herramientas de generación de código por ia: evaluación de calidad y análisis de desempeño. *Ciencia e Ingeniería: Revista de investigación interdisciplinar en biodiversidad y desarrollo sostenible, ciencia, tecnología e innovación y procesos productivos industriales*, 11(2):7. URL: <https://dialnet.unirioja.es/servlet/articulo?codigo=9685509>.
- [Nikhil Rao, 2023] Nikhil Rao, A. P. (2023). Reflex. URL: <https://reflex.dev/>.
- [NumPy,] NumPy. The fundamental package for scientific computing with python. URL: <https://numpy.org/>.

- [Oracle, 2021] Oracle (2021). What is a relational database? (rdbms)? URL: <https://www.oracle.com/database/what-is-a-relational-database/>.
- [Orgánica, 2018] Orgánica, L. (3/2018). Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales. URL: <https://www.boe.es/eli/es/lo/2018/12/05/3/con>.
- [OS,] OS, R. P. Raspberry pi os (previously called raspbian) is our official, supported operating system. URL: <https://www.raspberrypi.com/software/operating-systems/>.
- [Pandas,] Pandas. Pandas - python data analysis library. URL: <https://pandas.pydata.org/>.
- [PostgreSQL, 2026] PostgreSQL (2026). Postgresql: The world's most advanced open source relational database. URL: <https://www.postgresql.org/>.
- [Psycopg2-binary,] Psycopg2-binary. Postgresql database adapter for the python programming language. URL: <https://pypi.org/project/psycopg2-binary/>.
- [Python, a] Python. Python software foundation. URL: <https://www.python.org/downloads/>.
- [Python, b] Python, B. La biblioteca estándar de python. URL: <https://docs.python.org/es/3/library/index.html>.
- [Shanbehzadeh M, 2020] Shanbehzadeh M, K.-A. H. (2020). Using electronic data capture for cardiovascular electrophysiology invasive procedures: An important step towards interoperable clinical registries. *Frontiers in Health Informatics*, 9:48. Artículo que evidencia cómo la introducción manual de datos en registros clínicos (como REDCap) supone una alta carga de trabajo, consumo de tiempo y riesgo de error humano para los facultativos. URL: <https://healthinformaticsjournal.com/downloads/files/236.pdf>.
- [Siddiqui, 2025] Siddiqui, L. (2025). Coeficiente de determinación: Lo que nos dice el r cuadrado. URL: <https://www.datacamp.com/es/tutorial/coefficient-of-determination>.
- [SKlearn,] SKlearn. scikit-learn - machine learning in python. URL: <https://scikit-learn.org/stable/>.

- [Solé, 2021] Solé, R. (2021). Raspberry pi. URL: https://www.profesionalreview.com/2021/07/18/que-es-raspberry-pi/#:~:text=Table_title=%20Tabla%20comparativa%20Table_content:%20header:%20%7C%20MODELO,%7C%20CONECTIVIDAD%20INAL%C3%81MBRICA:%20802.11ac%20/%20Bluetooth%205.0.
- [Sommerville, 2005] Sommerville, I. (2005). *Ingeniería del software*. Pearson Educación, Madrid, 7 edition. URL: <https://ulagos.wordpress.com/wp-content/uploads/2010/07/ian-sommerville-ingenieria-de-software-7-ed.pdf>.
- [TexMaker,] TexMaker. Texmaker. URL: <https://www.xmlmath.net/texmaker/>.
- [Vite,] Vite. La herramienta de compilación para la web. URL: <https://es.vite.dev/>.
- [VSCode,] VSCode. Visual studio code - code editing. redefined. URL: <https://code.visualstudio.com/>.
- [Wikipedia, 2025a] Wikipedia (2025a). Computadora monoplaca. URL: https://es.wikipedia.org/wiki/Computadora_monoplaca.
- [Wikipedia, 2025b] Wikipedia (2025b). Sql. URL: <https://es.wikipedia.org/wiki/SQL>.